DREAM HOME RECOMMENDATION SYSTEM

A PROJECT REPORT

Submitted by

MBNP SAI DATTA BHAAVAN

(2116241401087)

in partial fulfilment of the End Semester Examination for the course

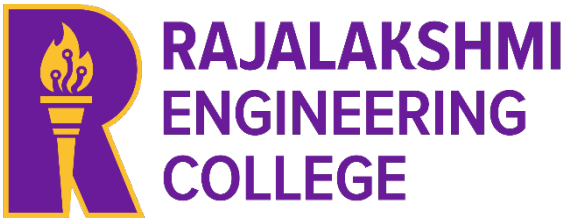
CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF TECHNOLOGY

COMPUTER SCIENCE AND BUSINESS SYSTEMS



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "DREAM HOME RECOMMENDATION SYSTEM" is the bonafide work of MBNP SAI DATTA BHAAVAN (2116241401087), who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE

Dr. K.R. Sowmia
HEAD OF THE DEPARTMENT
Department of Computer Science and Business Systems
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Mr. M.L. Alphin Ezhil Manuel
COURSE FACULTY
Assistant Professor
Department of Computer Science and Business Systems
Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on 03 November 2026.

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, MBNP SAI DATTA BHAAVAN (2116241401087), hereby declare that the project report titled “DREAM HOME RECOMMENDATION SYSTEM”, submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: Chennai
Date: 01 November 2026

SIGNATURE OF THE STUDENT
MBNP SAI DATTA BHAAVAN
2116241401087

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman Mr. S. Meganathan, B.E., F.I.E., Chairperson Dr. Thangam Meganathan, Ph.D., and Vice-Chairman Mr. Abhay Shankar Meganathan, B.E., M.S., for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, Dr. S.N. Murugesan, M.E., Ph.D., Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank Dr. K.R. Sowmia, Ph.D., Associate Professor and Head of the Department, Department of Computer Science and Business Systems, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, Mr. M.L. Alphin Ezhil Manuel, Assistant Professor, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation and the Zoho Creator Education Programme for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

MBNP SAI DATTA BHAAVAN

ABSTRACT

Choosing a home inside a gated community is one of the largest financial decisions a family makes, yet buyers usually compare options through brochures, broker advice and personal notes. Budget, monthly maintenance, location, builder reputation and everyday amenities such as security, water, parking, garbage handling, connectivity and power backup are weighed informally, and the buyer's own experience in the previous home is rarely used. The result is a subjective choice with no record of why one community was preferred over another.

This project develops the Dream Home Recommendation System on Zoho Creator, a low-code platform that provides forms, a managed database, reports, pages and workflow automation without building a separate software stack. The application has two forms, GatedCommnunity for the community catalogue and Buyer_Preference for the buyer's budget, location, dream features and ratings of the previous home, two list reports and one dashboard page, Buyer Interest Analysis, with a bar chart and a pie chart. A form workflow runs a Deluge script whenever a buyer record is saved. The script gives more weight to the factors that the buyer rated poorly in the previous home, filters communities by budget, maintenance and location, calculates an experience score and a dream-match score, adds a small bonus for the preferred area, and stores the best community, a final score out of five and a result of Highly Recommended, Consider or Not Recommended.

The application was exercised with sample buyer and community records, the behaviour of the script was observed on the stored records, and several data-consistency defects were logged for correction. Version 1.0 was published from the Development environment to the Stage and Production environments on 01 November 2026. The system replaces manual comparison with a repeatable and explainable score, and the dashboard gives an instant visual summary of buyer outcomes.

Keywords: *Low-Code Development; Zoho Creator; Deluge Scripting; Workflow Automation; Recommendation System; Real Estate*

PROJECT AT A GLANCE

Attribute	Details
Project Title	Dream Home Recommendation System
Application Domain	Real Estate / Home-Buyer Advisory
Student Name & Register No.	MBNP SAI DATTA BHAAVAN – 2116241401087
Zoho Creator Edition	Zoho Creator (trial edition)
Application Name (Link Name)	Dream_Home_Recommendation_System (dream-home-recommendation-system)
Published / Live URL	https://creatorapp.zoho.in/ saidattabhaavan.mbn.20241618/environment/ development/dream-home-recommendation-system/
Demo Credentials (per role)	Administrator: saidattabhaavan.mbn.2024.csbs@rajalakshmi.edu.in (password provided separately to the examiner)
Number of Forms / Subforms	2 / 0
Number of Reports (by type)	List: 2 (Buyer_Preference Report, GatedCommnunity Report) Kanban: 0 Calendar: 0 Chart: 2 charts on the dashboard page Others: 0
Number of Pages / Dashboards	1 (Buyer Interest Analysis)
Number of Workflows	Form: 1 Schedule: 0 Approval: 0 Blueprint: 0 Batch: 0
Deluge Functions / Custom Actions	1 Deluge script (DS-01); 0 standalone functions
Integrations	None in version 1.0 (all logic runs inside Zoho Creator)
Roles & Profiles Defined	Administrator (application owner); no additional roles configured
Environments Used	Development → Stage → Production (version 1.0)
Date of Production Deployment	01 November 2026

TABLE OF CONTENTS

ABSTRACT....iv
PROJECT AT A GLANCE....v
LIST OF TABLES....ix
LIST OF FIGURES....x
LIST OF ABBREVIATIONS....xi
INTRODUCTION....1
1.1 Background....1
1.2 Problem Statement....1
1.3 Objectives....2
1.4 Scope of the Project....2
1.5 Justification for a Low-Code Approach....3
1.6 Organisation of the Report....3
LITERATURE SURVEY AND SYSTEM STUDY....4
2.1 Study of the Existing System....4
2.2 Limitations of the Existing System....4
2.3 Low-Code versus Traditional Development....5
2.4 Literature Survey....5
2.5 Emerging Trends in Low-Code Development....7
2.6 Proposed System....7
REQUIREMENT ANALYSIS....8
3.1 Stakeholders and User Roles....8
3.2 Functional Requirements....8
3.3 Non-Functional Requirements....9
3.4 Hardware and Software Requirements....10
3.5 Use Case Model....10
SYSTEM DESIGN....12
4.1 System Architecture....12
4.2 Data Model....12
4.3 Form Design....14
4.4 Process and Workflow Design....16
4.5 User Interface and Navigation Design....17
4.6 Access Control Design....18
IMPLEMENTATION....20
5.1 Application Setup....20
5.2 Forms and Data Capture....20

5.3 Reports....26
5.4 Pages and Dashboards....27
5.5 Deluge Scripting....28
5.6 Workflows and Automation....31
5.7 Integrations....32
5.8 User Management and Role-Based Access....32
5.9 AI and Analytics Features....32
5.10 Pro-Code Extension....32
5.11 Mobile and Portal Access....33
TESTING AND DEBUGGING....34
6.1 Testing Strategy....34
6.2 Test Cases....34
6.3 Debugging Deluge Scripts....36
6.4 Defect Log....37
6.5 User Acceptance Testing....38
DEPLOYMENT, SECURITY AND GOVERNANCE....39
7.1 Environments and Deployment Strategy....39
7.2 Performance Optimisation....40
7.3 Security Features....41
7.4 Governance and Administration....41
RESULTS AND DISCUSSION....43
8.1 Application Output....43
8.2 Achievement of Objectives....44
8.3 Comparison with the Existing System....45
8.4 Limitations of the Low-Code Approach....45
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum....45
CONCLUSION AND FUTURE ENHANCEMENTS....47
9.1 Conclusion....47
9.2 Future Enhancements....47
9.3 Learning Outcomes and Career Relevance....47
REFERENCES....49
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST....51
APPENDIX B – COURSE OUTCOME MAPPING....54
APPENDIX C – COMPLETE DELUGE SOURCE CODE....56
APPENDIX D – USER MANUAL....59
D.1 Accessing the Application....59

D.2 Instructions for the Administrator / Advisor – Maintaining the Community Catalogue.....59
D.3 Instructions for Recording a Buyer and Reading the Recommendation.....59

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application.....5
Table 2.2: Summary of Literature Survey.....6
Table 3.1: Stakeholders and User Roles.....8
Table 3.2: Functional Requirements.....8
Table 3.3: Non-Functional Requirements.....9
Table 3.4: Hardware and Software Requirements.....10
Table 3.5: Use Case Description – Enter Buyer Preferences and Obtain a Recommendation.....11
Table 4.1: Entities and Relationships.....13
Table 4.2: Field Specification – Buyer_Preference Form.....14
Table 4.3: Field Specification – GatedCommnunity Form.....15
Table 4.4: Role–Permission Matrix.....18
Table 5.1: Reports Implemented.....26
Table 5.2: Deluge Scripts Implemented.....28
Table 5.3: Workflows Implemented.....31
Table 5.4: Integrations Implemented.....32
Table 6.1: Test Cases.....34
Table 6.2: Defect Log.....37
Table 6.3: User Acceptance Feedback.....38
Table 7.1: Deployment Checklist.....40
Table 7.2: Performance Optimisation Results.....40
Table 7.3: Security Controls Implemented.....41
Table 8.1: Mapping of Objectives to Outcomes.....44
Table 8.2: Existing System versus Proposed Application.....45
Table 9.1: Feature Coverage Checklist.....51
Table 9.2: Mapping of Course Outcomes to Report Chapters.....54

LIST OF FIGURES

Figure 2.1: As-Is Process Flow of the Existing Home-Selection Process.....4
Figure 3.1: Use Case Diagram of the Dream Home Recommendation System.....10
Figure 4.1: System Architecture of the Dream Home Recommendation System……12
Figure 4.2: Entity–Relationship Diagram of the Application.....13
Figure 4.3: Workflow / Process Flow for Calculating a Recommendation.....17
Figure 4.4: Navigation Structure and Wireframe of the Buyer Interest Analysis Page 18
Figure 5.1: Application Home and Navigation Sections.....20
Figure 5.2: Buyer_Preference Form in the Form Builder.....23
Figure 5.3: GatedCommnunity Form in the Form Builder.....24
Figure 5.4: GatedCommnunity Form in Live Mode.....25
Figure 5.5: Buyer_Preference Report with the Record Detail Panel.....27
Figure 5.6: Buyer Interest Analysis Page (Bar Chart of Prices and Pie Chart of Results).....28
Figure 5.7: Calculate Recommendation Workflow in the Deluge Editor.....31
Figure 7.1: Environments and Publish to Stage.....39
Figure 7.2: Component Change Log in the Publish to Stage View.....42
Figure 8.1: Scenario Step 1 – Buyer Preferences and Previous-Home Ratings Stored in the Report.....43
Figure 8.2: Scenario Step 2 – Result Fields Written by the Workflow (Final_score 0.00, Not Recommended).....44

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
BHK	Bedroom, Hall and Kitchen
CRUD	Create, Read, Update, Delete
DS	Deluge Script (script identifier, e.g., DS-01)
ER	Entity–Relationship
FR	Functional Requirement
LCAP	Low-Code Application Platform
NFR	Non-Functional Requirement
RBAC	Role-Based Access Control
RERA	Real Estate (Regulation and Development) Act
SDLC	Software Development Life Cycle
UAT	User Acceptance Testing
UC	Use Case
UI	User Interface
Zia	Zoho’s AI assistant

CHAPTER 1
INTRODUCTION

1.1 Background

Buying a flat or villa in a gated community is usually the largest single purchase a family makes. A typical metropolitan market such as Chennai offers dozens of gated communities with different prices, monthly maintenance charges, possession status, builder reputation and amenities such as security, water supply, parking, garbage handling, internet connectivity and power backup. India's Real Estate (Regulation and Development) Act, 2016 has made project information more transparent for buyers [1], but choosing between the available options is still left to the buyer.

This project considers a realistic home-buyer advisory scenario. An advisor maintains a catalogue of gated communities, and each prospective buyer states a budget, a limit on monthly maintenance, a preferred location, the kind of home dreamed of (BHK, possession, features and surroundings) and a rating of the home in which the buyer currently lives. At present this information is kept in notebooks, messages and spreadsheets, and the final shortlist is produced by judgement rather than by a repeatable method.

The Dream Home Recommendation System digitises this process on Zoho Creator. The community catalogue and the buyer's preferences are stored in two forms; a workflow evaluates every community against the buyer's requirements and stores the best match together with a score and a result.

1.2 Problem Statement

Home buyers and advisors have no structured, repeatable way of comparing gated communities against a buyer's budget, location and lifestyle needs. Important lessons from the buyer's previous home, such as poor water supply or limited parking, are not converted into priorities, and the reasoning behind a recommendation is not recorded.

- Community details are scattered across brochures, messages and spreadsheets, so information is duplicated, outdated or incomplete.
- Budget and monthly-maintenance limits are checked by hand, so unsuitable options are shortlisted and suitable ones are missed.

- The buyer’s experience in the previous home is not used to decide which amenities matter most.
- Comparison is subjective; there is no score, no result category and no record of why a community was recommended.
- Management has no consolidated view of how many buyers fall into each recommendation category or how communities are priced.

1.3 Objectives

- To design a data model and two forms, GatedCommnunity and Buyer_Preference, that capture community details and buyer requirements with consistent drop-down values.
- To automate the recommendation using a Deluge script in a form workflow that filters communities by budget, maintenance and location and calculates an experience score and a dream-match score.
- To store the best-matching community, the final score and a result category (Highly Recommended, Consider or Not Recommended) on every buyer record.
- To provide list reports and a dashboard page with charts that summarise community prices and recommendation results.
- To deploy version 1.0 of the application through the Development, Stage and Production environments.
- To test the application with sample data, record the defects found and document how they can be corrected.

1.4 Scope of the Project

In scope: maintaining a catalogue of gated communities (location, price, maintenance, BHK, possession, ratings of eight amenity factors, surroundings and features); capturing a buyer’s budget, maintenance limit, location preference, dream home requirements and previous-home ratings; automatic calculation of the best-matching community, final score and result; two list reports; one dashboard page with a bar chart and a pie chart; and access through the web (the Zoho Creator mobile app can open the same application).

Out of scope: integration with property portals or maps, e-mail or SMS notifications, payment or booking of a unit, document verification, a public buyer portal with separate

login, and approval workflows. The ratings stored in the catalogue are sample values entered for demonstration and are not an assessment of any real builder or project.

1.5 Justification for a Low-Code Approach

The application is a typical departmental business application: two forms with clear fields, one rule-based calculation, a few reports and a dashboard, used by non-technical staff. Building it with traditional code would need a database, a back-end service, a web front-end, hosting and separate mobile support. Zoho Creator provides all of these as managed services, the form, report and page builders are visual, and Deluge, Zoho's scripting language, is needed only for the scoring logic [2], [3].

This shortens development from weeks to days, allows a citizen developer such as an advisor to maintain the application, and gives built-in hosting, versioned environments and mobile access. Low-code platforms are widely recognised as suitable for exactly this class of data-centric business application [4], [5].

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related low-code literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance optimisation, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

The current process is manual. A buyer hears about projects from brokers, advertisements and friends, visits builder websites and property portals, and collects brochures. Price, maintenance and amenities are noted in a notebook or spreadsheet. Options are then compared by hand, usually with the opinion of family and friends, and a decision is taken on incomplete and subjective information. Problems with water, parking or connectivity are often discovered only after moving in. Figure 2.1 shows this as-is process.

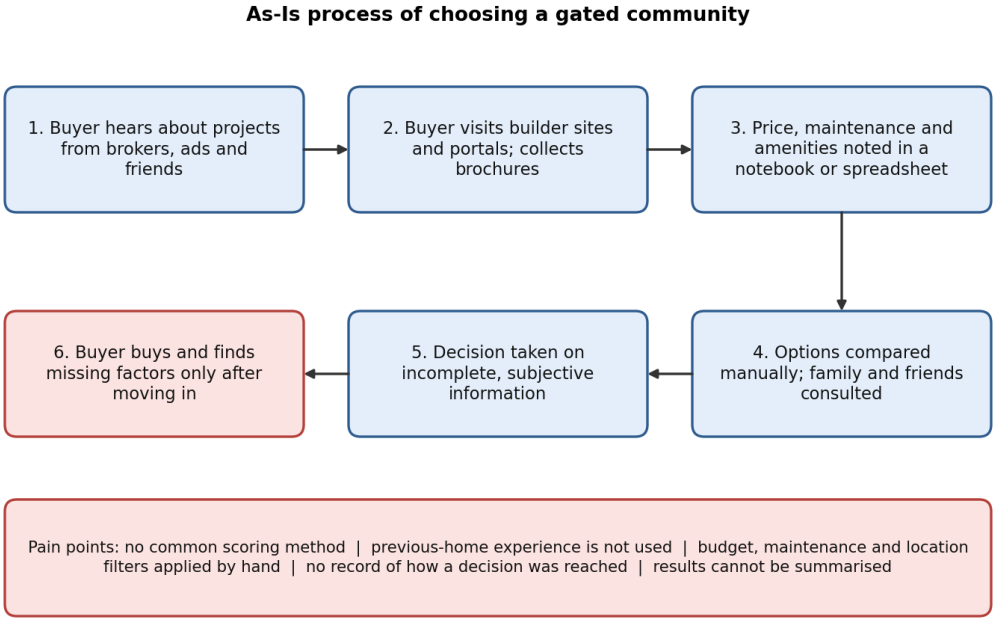


Figure 2.1: As-Is Process Flow of the Existing Home-Selection Process

2.2 Limitations of the Existing System

- Information is collected from many sources and written down by hand, so it is incomplete and quickly outdated.
- Budget, maintenance and location limits are checked manually, which is slow and error-prone.
- The buyer’s experience in the previous home is not converted into priorities for the new home.

- There is no common scoring method, so two advisors can recommend different communities for the same buyer.
- No record of the reasoning is kept and no summary of outcomes can be produced.

2.3 Low-Code versus Traditional Development

Table 2.1 compares the two approaches for this application. The effort figures are the author's estimates for an application of this size.

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time	6–8 weeks for database, back-end, front-end, scoring service and deployment	1–2 weeks for forms, reports, dashboard and one workflow
Skills required	Full-stack skills: SQL, a back-end language, HTML/CSS/JavaScript, DevOps	Zoho Creator builders and basic Deluge scripting
Hosting & infrastructure	Server or cloud hosting, database administration, backups	Hosted and managed by Zoho
Mobile application	Separate responsive or native app development	Zoho Creator mobile app can open the same application
Security & access control	Login, sessions and role checks coded by hand	Built-in Zoho login, users, roles and permission profiles
Maintenance & change	Code changes, testing and redeployment	Fields, reports and scripts edited visually; versions published through environments
Customisation limits	Unlimited	Limited to platform features; complex UI or heavy computation needs widgets or pro-code
Cost	Developer time plus hosting	Subscription (trial edition used for this project)

2.4 Literature Survey

Six sources were reviewed: three on low-code platforms, two on recommendation and multi-criteria decision methods, and the official Zoho documentation, together with the real-estate regulation that motivates transparent information for buyers.

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	A. Sahay, A. Indamutsa, D. Di Ruscio and A. Pierantonio, IEEE/Euromicro SEAA, 2020 [4]	Supporting the understanding and comparison of low-code development platforms	Compares major low-code platforms and their common features: visual modelling, data management, workflow and integration.	Guided the choice of Zoho Creator features used for forms, workflow and reporting.
2	A. C. Bock and U. Frank, Business & Information Systems Engineering, 2021 [5]	Low-Code Platform	Defines low-code platforms and their typical use for departmental business applications built by citizen developers.	Supports the justification for a low-code approach for an advisory team.
3	D. Di Ruscio et al., Software and Systems Modeling, 2022 [6]	Low-code development and model-driven engineering: Two sides of the same coin?	Relates low-code platforms to model-driven engineering and discusses their limitations.	Used in the discussion of low-code limits and the pro-code spectrum.
4	F. Ricci, L. Rokach and B. Shapira (Eds.), Springer, 2015 [7]	Recommender Systems Handbook (2nd ed.)	Describes content-based and knowledge-based recommenders that match item attributes to stated user requirements.	Basis for matching community attributes to the buyer's stated requirements.
5	R. L. Keeney and H. Raiffa, Cambridge University Press, 1993 [8]	Decisions with Multiple Objectives: Preferences and Value Tradeoffs	Presents weighted additive value models for choosing between alternatives with several criteria.	Basis for weighting the eight amenity factors by importance and combining the scores.
6	Zoho Corporation, online documentation [2], [3]	Zoho Creator Help and Deluge Reference	Documents forms, reports, pages, workflows, fetch statements, loops and environments.	Primary reference for implementing every component of the application.
7	Government of India, 2016 [1]	The Real Estate (Regulation and Development) Act, 2016	Promotes transparency in real-estate projects and the information available to buyers.	Motivates a transparent, recorded basis for recommending a community.

2.5 Emerging Trends in Low-Code Development

Several current trends shaped the design. AI-assisted application building and Zia features show how low-code platforms are moving towards generating forms and insights automatically; this project uses the visual builders and keeps AI features as a future enhancement. Citizen development [9] is reflected in keeping the application simple enough for an advisor to maintain. Hyper-automation is reflected in triggering the complete evaluation from a single form submission, and composable applications are a natural next step, for example by reusing Zoho Mail or Zoho CRM through connections instead of rebuilding them.

2.6 Proposed System

The proposed system replaces notebooks and spreadsheets with one Zoho Creator application. The community catalogue is entered once in the GatedCommnunity form. A buyer’s requirements and previous-home ratings are entered in the Buyer_Preference form. When the buyer record is saved, a Deluge script evaluates every community, discards those that exceed the budget, the maintenance limit or the location preference, scores the rest and stores the best community, the final score and a result category on the buyer record. The two list reports and the Buyer Interest Analysis dashboard give instant summaries. Each limitation listed in Section 2.2 is therefore addressed: data is stored once, filters are automatic, previous-home experience drives the weights, the score is repeatable and the outcome is recorded.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Administrator / Advisor	Real-estate advisor who owns the application and maintains the catalogue of gated communities.	Add and edit community records, enter buyer preferences, view reports and the dashboard, publish the application.
Home Buyer	Prospective buyer whose budget, location preference, dream features and previous-home ratings drive the recommendation.	Provide preferences and ratings; receive the recommended community, final score and result. In version 1.0 this is done through the Administrator login.

3.2 Functional Requirements

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall store each gated community with location, price, monthly maintenance, BHK (mandatory), possession and surroundings.	Administrator	High
FR-02	The system shall store ratings from 1 to 5 for eight factors (security, water, parking, garbage, interior support, connectivity, power backup and builder reputation) for each community.	Administrator	High
FR-03	The system shall capture a buyer’s budget, maximum maintenance, home city and state, preferred area, dream BHK, possession preference and location preference.	Administrator, Home Buyer	High
FR-04	The system shall capture the buyer’s ratings of the previous home for the same eight factors and the years lived there.	Administrator, Home Buyer	High
FR-05	The system shall consider only communities whose price and maintenance do not exceed the buyer’s limits.	Automatic	High
FR-06	The system shall apply the location preference: Nearby (same city), Same state or Other state (target state).	Automatic	High
FR-07	The system shall give each factor an importance of 6 minus the previous-home rating (blank counts as 3) and compute a weighted experience score.	Automatic	High

FR ID	Requirement	Role(s)	Priority
FR-08	The system shall compute a dream-match score from BHK, possession, preferred features and surroundings, and add a bonus of 0.3 when the preferred area matches.	Automatic	High
FR-09	The system shall store the best-matching community, the final score (rounded to two decimals, maximum 5) and a result: Highly Recommended (3.75 or more), Consider (3.0 or more) or Not Recommended.	Automatic	High
FR-10	The system shall set the score to 0 and the result to Not Recommended when no community satisfies the filters.	Automatic	High
FR-11	The system shall provide list reports of buyer and community records with a record detail view.	Administrator	Medium
FR-12	The system shall provide a dashboard with a bar chart of community prices and a pie chart of recommendation results.	Administrator	Medium
FR-13	The system shall be released through Development, Stage and Production environments with a version number.	Administrator	Medium

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	The recommendation shall be calculated and stored when the buyer record is saved, for a catalogue of up to a few hundred communities.
NFR-02	Security	Only authenticated Zoho users shall access the application; the application shall not store passwords or API keys.
NFR-03	Usability	Choice lists shall be used wherever possible so that values are consistent; the application shall be usable in a web browser.
NFR-04	Availability	The application shall be hosted and maintained by Zoho Creator.
NFR-05	Maintainability	Scoring weights and thresholds shall be visible in one script so that they can be changed by a citizen developer.
NFR-06	Auditability	Component changes shall be traceable through the publish change log, and record creation details through system fields.

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator (trial edition), web-based
Scripting language	Deluge (Data Enriched Language for the Universal Grid Environment)
Browser	Google Chrome / Microsoft Edge / Mozilla Firefox (latest version)
API testing tool	Not required: no external API is called in version 1.0
Other Zoho services	None in version 1.0
Mobile	Zoho Creator mobile app (Android / iOS) can open the application; not used for the demonstration
Client hardware	Intel Core i3/i5 or above, 8 GB RAM, internet connection

3.5 Use Case Model

Figure 3.1 shows the use cases. The Administrator maintains the catalogue, views reports and the dashboard and publishes the application. The Home Buyer supplies preferences and receives the recommendation; the calculation itself is automatic and is included in the preference-entry use case.

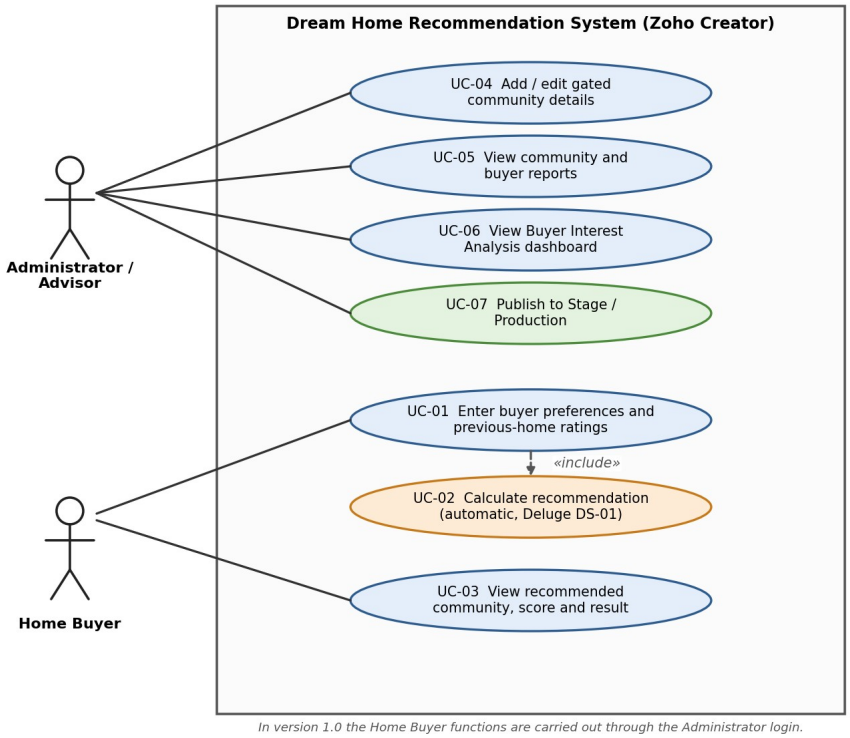


Figure 3.1: Use Case Diagram of the Dream Home Recommendation System

Table 3.5: Use Case Description – Enter Buyer Preferences and Obtain a Recommendation

Use Case ID	[UC-01]
Use Case ID	UC-01
Use Case Name	Enter buyer preferences and obtain a recommendation
Actor(s)	Administrator / Advisor (on behalf of the Home Buyer)
Pre-condition	The user is logged in to Zoho Creator and the GatedCommnunity catalogue contains at least one community.
Main Flow	1. The user opens the Buyer_Preference form. 2. The user enters the buyer name, maximum budget, maximum maintenance, home state and city, preferred area, dream BHK, possession and location preference. 3. The user enters the previous-home ratings, years lived, surroundings and features. 4. The user submits the form. 5. The workflow Calculate Recommendation runs DS-01. 6. The best community, final score and result are stored on the record. 7. The user opens the record in the Buyer_Preference Report to read the result.
Alternate Flow	5a. No community satisfies the budget, maintenance and location filters: the score is set to 0 and the result to Not Recommended. 5b. Previous-home ratings are blank: the value 3 is used for each blank rating.
Post-condition	The buyer record holds the recommended community, final score and result, and the dashboard reflects the new result.
Related FR	FR-03 to FR-10

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

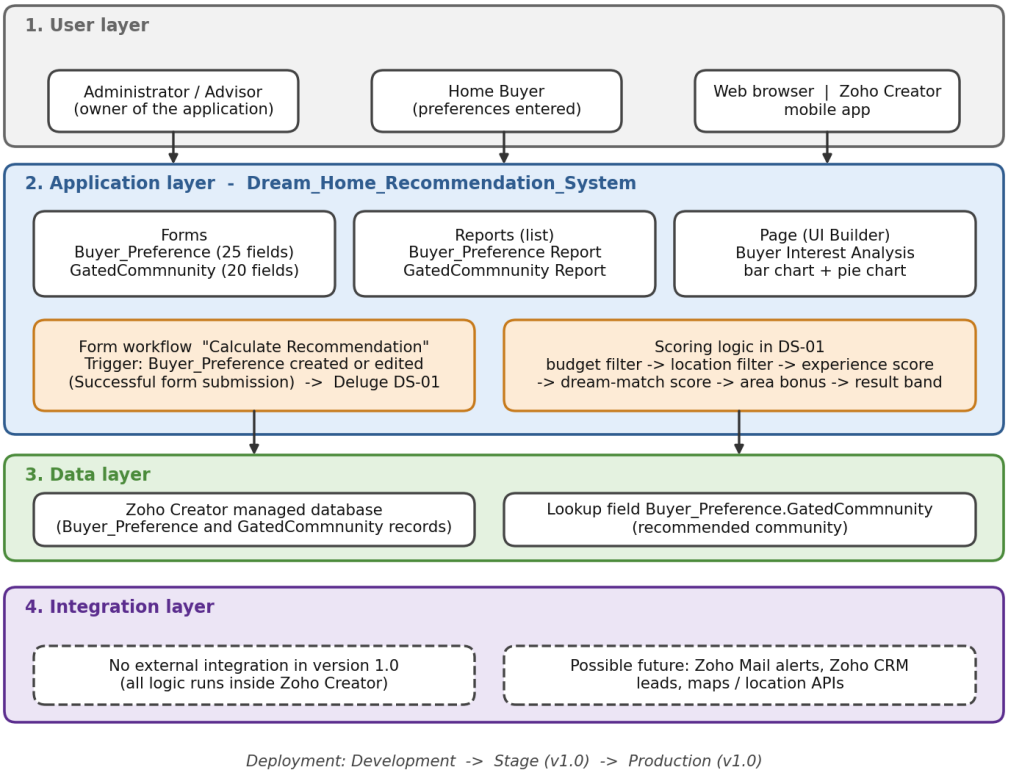


Figure 4.1: System Architecture of the Dream Home Recommendation System

The architecture has four layers. The user layer contains the Administrator or Advisor and the Home Buyer, who reach the application through a web browser or the Zoho Creator mobile app. The application layer is the Zoho Creator application Dream_Home_Recommendation_System: the Buyer_Preference and GatedCommnunity forms for data capture, a list report for each form, the Buyer Interest Analysis page for analytics, and the form workflow Calculate Recommendation, which runs the Deluge script DS-01. The data layer is Zoho Creator's managed database, which stores the records and the lookup field that links a buyer to the recommended community. The integration layer is empty in version 1.0 because all logic runs inside Zoho Creator; Zoho Mail, Zoho CRM or location services can be added later through connections.

4.2 Data Model

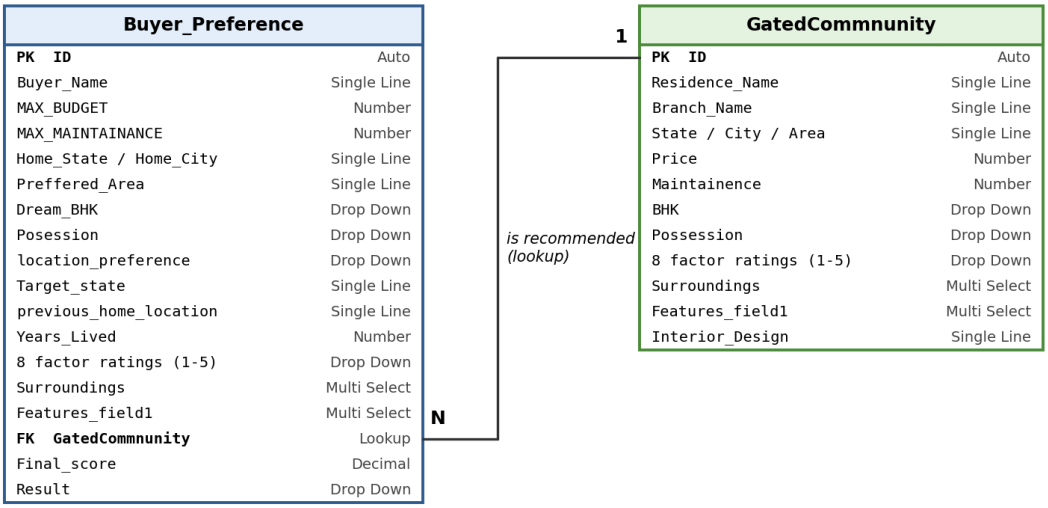


Figure 4.2: Entity–Relationship Diagram of the Application

The application has two entities. GatedCommnunity holds the catalogue of communities, and Buyer_Preference holds each buyer's requirements and the calculated result. The relationship is many-to-one: many buyer records can point to the same community, while each buyer record points to at most one community, the best match. It is implemented with the lookup field GatedCommnunity on the Buyer_Preference form. The two forms also share the same eight rating factors, which is what allows the script to compare a buyer's previous home with each community factor by factor.

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Buyer_Preference	ID, Buyer_Name, MAX_BUDGET, MAX_MAINTAINANCE, Home_City, location_preference, Final_score, Result	GatedCommunity	Many-to-One	Lookup field GatedCommnunity
GatedCommnunity	ID, Residence_Name, State, City, Area, Price, Maintainence, BHK	Buyer_Preference	One-to-Many	Referenced by the lookup field

4.3 Form Design

Table 4.2 lists the twenty-five fields of the Buyer_Preference form and Table 4.3 lists the twenty fields of the GatedCommnunity form. Field labels and link names are identical and are reproduced exactly as they appear in Zoho Creator, including the original spelling of Preffered_Area, Posession, MAX_MAINTAINANCE and GatedCommnunity.

Table 4.2: Field Specification – Buyer_Preference Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Buyer_Name	Buyer_Name	Single Line	No	Max 255 characters	Name of the buyer
MAX_BUDGET	MAX_BUDGET	Number	No	—	Budget filter (price ≤ budget)
MAX_MAINTAINANCE	MAX_MAINTAINANCE	Number	No	—	Maintenance filter
Home_State	Home_State	Single Line	No	—	Used by “Same state”
Home_City	Home_City	Single Line	No	—	Used by “Nearby”
Preffered_Area	Preffered_Area	Single Line	No	—	+0.3 bonus if equal to community Area
Dream_BHK	Dream_BHK	Drop Down	No	Choice list (e.g., 4BHK)	Compared with community BHK
Posession	Posession	Drop Down	No	Choice list (e.g., Ready to move, Any)	“Any” matches all
location_preference	location_preference	Drop Down	No	Nearby / Same state / Other state	Selects the location filter
Target_state	Target_state	Single Line	No	—	Used by “Other state”
previous_home_location	previous_home_location	Single Line	No	—	Information only
Years_Lived	Years_Lived	Number	No	—	Information only
Security	Security	Drop Down	No	Rating 1–5; blank counts as 3	Previous-home rating
Water	Water	Drop Down	No	Rating 1–5; blank counts as 3	Previous-home rating

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Parking	Parking	Drop Down	No	Rating 1–5; blank counts as 3	Previous-home rating
Garbage	Garbage	Drop Down	No	Rating 1–5; blank counts as 3	Previous-home rating
Interior_Support	Interior_Support	Drop Down	No	Rating 1–5; blank counts as 3	Previous-home rating
Connectivity	Connectivity	Drop Down	No	Rating 1–5; blank counts as 3	Previous-home rating
Builder_Reputation	Builder_Reputation	Drop Down	No	Rating 1–5; blank counts as 3	Previous-home rating
Power_Backup	Power_Backup	Drop Down	No	Rating 1–5; blank counts as 3	Previous-home rating
Surroundings	Surroundings	Multi Select	No	Choice list	Share of selected items found in community
Features_field1	Features_field1	Multi Select	No	Choice list	Share of selected items found in community
GatedCommnunity	GatedCommnunity	Lookup	No	Lookup to GatedCommnunity	Set by DS-01
Final_score	Final_score	Decimal	No	2 decimal places	Set by DS-01
Result	Result	Drop Down	No	Highly Recommended / Consider / Not Recommended	Set by DS-01

Table 4.3: Field Specification – GatedCommnunity Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Residence_Name	Residence_Name	Single Line	No	Max 255 characters	Name of the community
Branch_Name	Branch_Name	Single Line	No	—	Project / branch name
State	State	Single Line	No	—	Compared by “Same state”, “Other state”
City	City	Single Line	No	—	Compared by “Nearby”
Area	Area	Single Line	No	—	Compared with Preffered_Area

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Price	Price	Number	No	—	Budget filter
Maintainence	Maintainence	Number	No	—	Maintenance filter
BHK	BHK	Drop Down	Yes	Choice list (e.g., 4BHK)	Compared with Dream_BHK
Possession	Possession	Drop Down	No	Choice list (e.g., Ready to move)	Compared with Posession
Security	Security	Drop Down	No	Rating 1–5; blank counts as 3	Community rating
Water	Water	Drop Down	No	Rating 1–5; blank counts as 3	Community rating
Parking	Parking	Drop Down	No	Rating 1–5; blank counts as 3	Community rating
Garbage	Garbage	Drop Down	No	Rating 1–5; blank counts as 3	Community rating
Interior_Support	Interior_Support	Drop Down	No	Rating 1–5; blank counts as 3	Community rating
Connectivity	Connectivity	Drop Down	No	Rating 1–5; blank counts as 3	Community rating
Power_Backup	Power_Backup	Drop Down	No	Rating 1–5; blank counts as 3	Community rating
Builder_Reputation	Builder_Reputation	Drop Down	No	Rating 1–5; blank counts as 3	Community rating
Surroundings	Surroundings	Multi Select	No	Choice list	Matched with buyer choices
Features_field1	Features_field1	Multi Select	No	Choice list	Matched with buyer choices
Interior_Design	Interior_Design	Single Line	No	—	Information only

Six field types are used: Single Line, Number, Drop Down, Multi Select, Lookup and Decimal. Only the BHK field of GatedCommnunity is mandatory; the other fields are optional so that a record can be saved with partial data, and DS-01 treats a blank rating as 3.

4.4 Process and Workflow Design

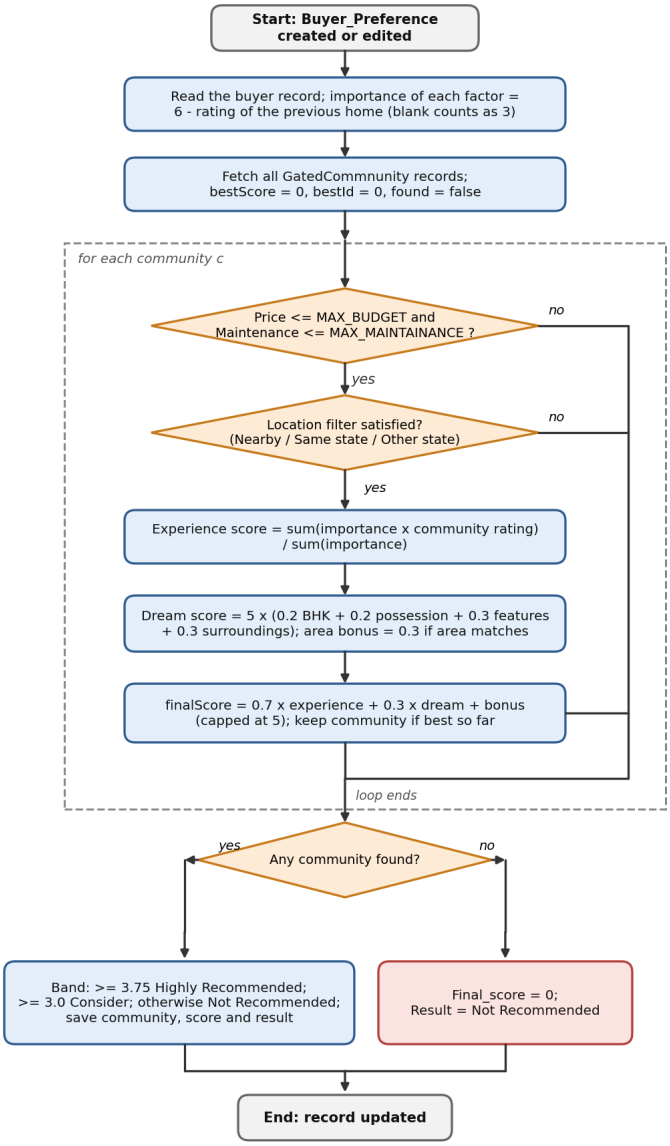


Figure 4.3: Workflow / Process Flow for Calculating a Recommendation

Figure 4.3 shows how the single workflow runs. When a Buyer_Preference record is created or edited and the submission succeeds, the script reads the buyer record, converts each previous-home rating into an importance weight, and loops over all communities. A community that exceeds the budget or maintenance limit, or fails the location rule, is skipped. Every remaining community receives an experience score, a dream-match score and an optional area bonus, which are combined into a final score; the best-scoring community is remembered. After the loop the script stores either the best community with a result band or, if none qualified, a score of 0 and the result Not Recommended.

4.5 User Interface and Navigation Design

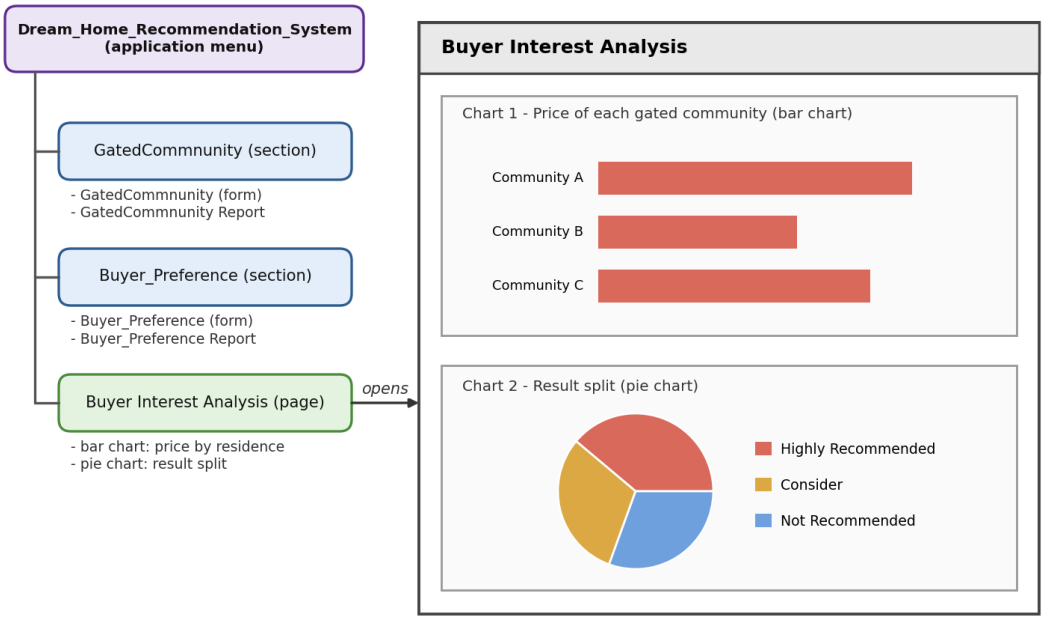


Figure 4.4: Navigation Structure and Wireframe of the Buyer Interest Analysis Page

The navigation is intentionally short. One section holds the GatedCommnunity form and its report, a second section holds the Buyer_Preference form and its report, and the Buyer Interest Analysis page is placed after them. The page contains two charts: a bar chart that compares the price of each community and a pie chart that shows how buyers are distributed across the result categories.

4.6 Access Control Design

Version 1.0 is operated through the Administrator account of the application owner. Table 4.4 records the permission design: the Administrator has full access, and the Home Buyer column is the planned design for a future buyer profile (A = Add, E = Edit, V = View, D = Delete, — = No access).

Table 4.4: Role–Permission Matrix

Component	Administrator (implemented)	Home Buyer (planned)
Form: GatedCommnunity	A E V D	—
Form: Buyer_Preference	A E V D	A V (own)
Fields: GatedCommnunity, Final_score, Result	V (set by DS-01)	V (own)
Report: GatedCommnunity Report	V E D	—
Report: Buyer_Preference Report	V E D	V (own)
Page: Buyer Interest Analysis	V	—

Component	Administrator (implemented)	Home Buyer (planned)
Publish to Stage / Production	Yes	—

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

The application was created in the Zoho Creator workspace of the author's trial account and is named Dream_Home_Recommendation_System. It was built in the Development environment with the form, report and page builders. The web menu contains the GatedCommnunity section (the form and its report), the Buyer_Preference section (the form and its report) and the page Buyer Interest Analysis (Figure 5.1). Figure 5.1 also shows the label "Development" beside the application name, which identifies the environment in which the screenshots were taken.

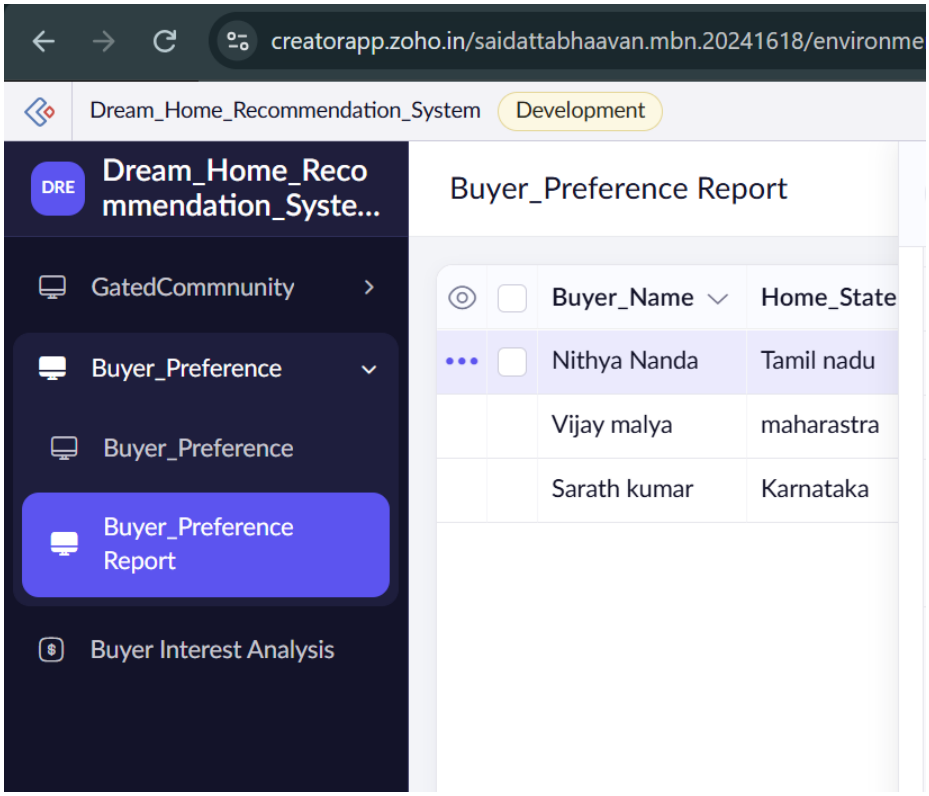
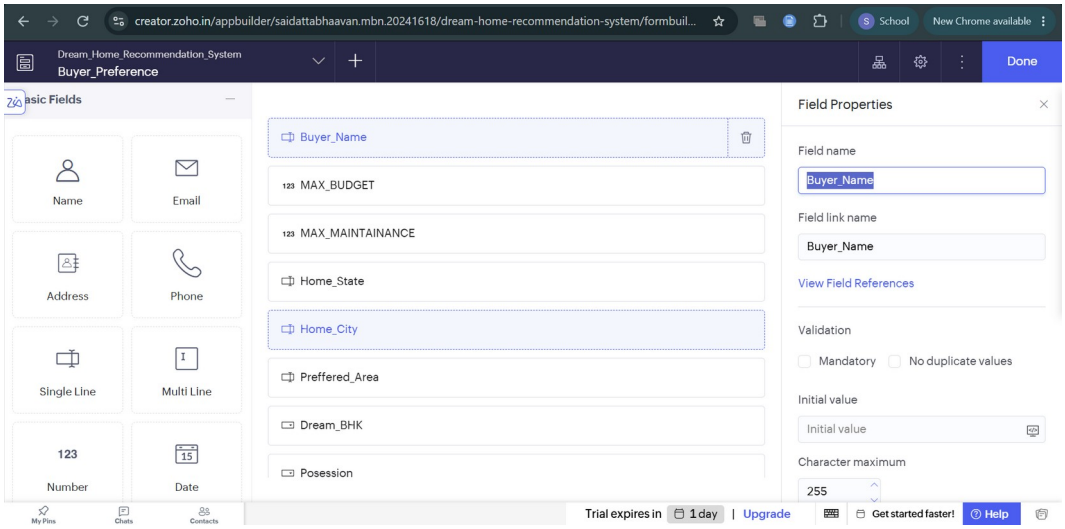


Figure 5.1: Application Home and Navigation Sections

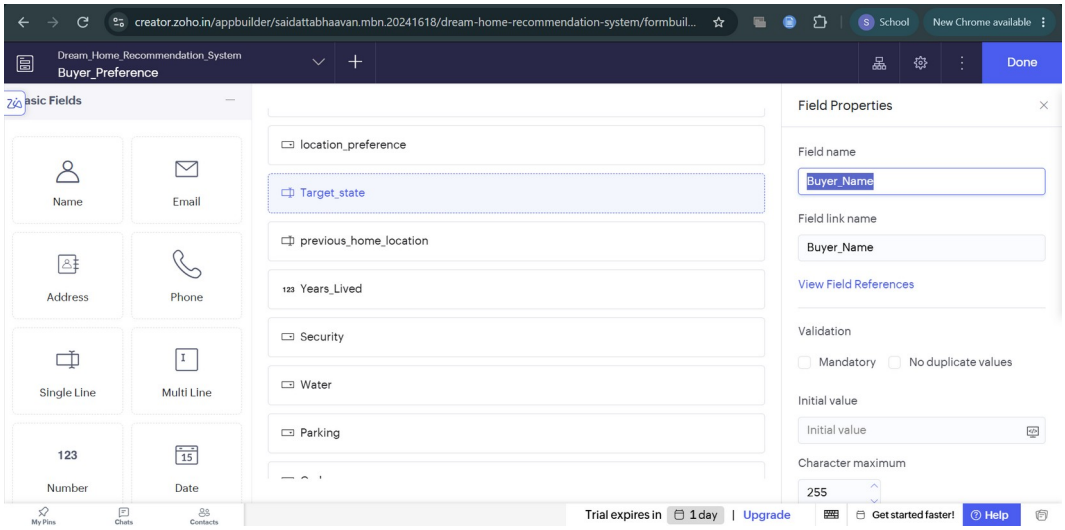
5.2 Forms and Data Capture

The application has two forms. GatedCommnunity stores the catalogue of gated communities and Buyer_Preference stores each buyer's requirements and the calculated recommendation. Both forms use the same eight rating fields (Security, Water, Parking, Garbage, Interior_Support, Connectivity, Power_Backup and Builder_Reputation), implemented as drop-down fields, so that a buyer's previous

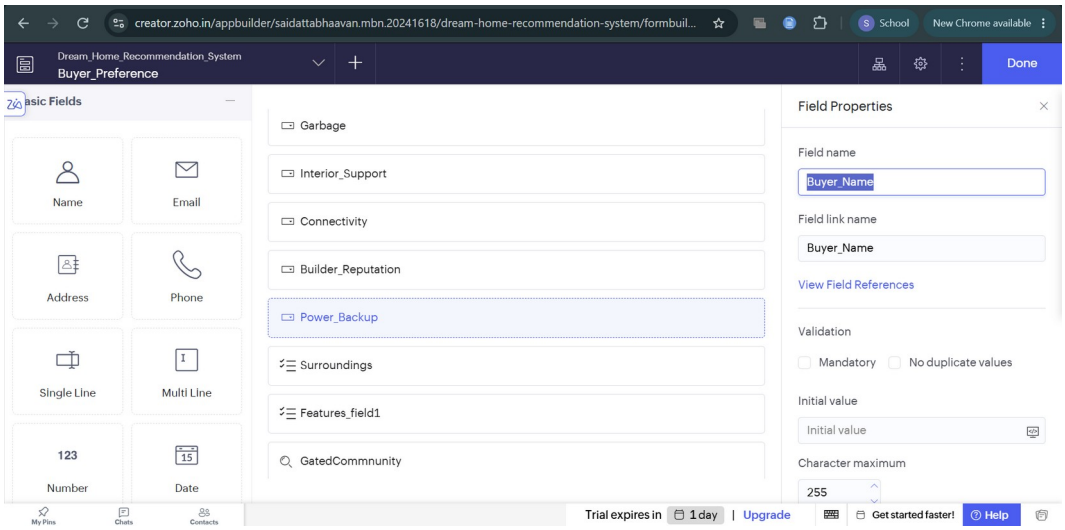
home and every community can be compared factor by factor. Surroundings and Features_field1 are multi-select fields, the location preference is a drop-down with the values Nearby, Same state and Other state, and the lookup field GatedCommnunity on the Buyer_Preference form holds the recommended community. Final_score (a decimal field) and Result (a drop-down) are filled by the workflow. The field lists are given in Tables 4.2 and 4.3.



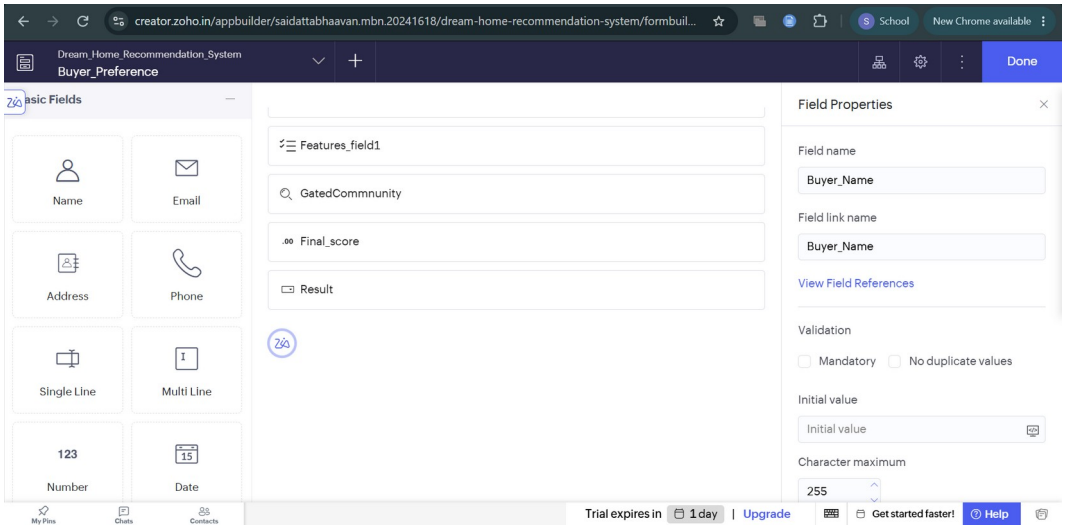
(a) Buyer_Name to Posession



(b) location_preference to Parking

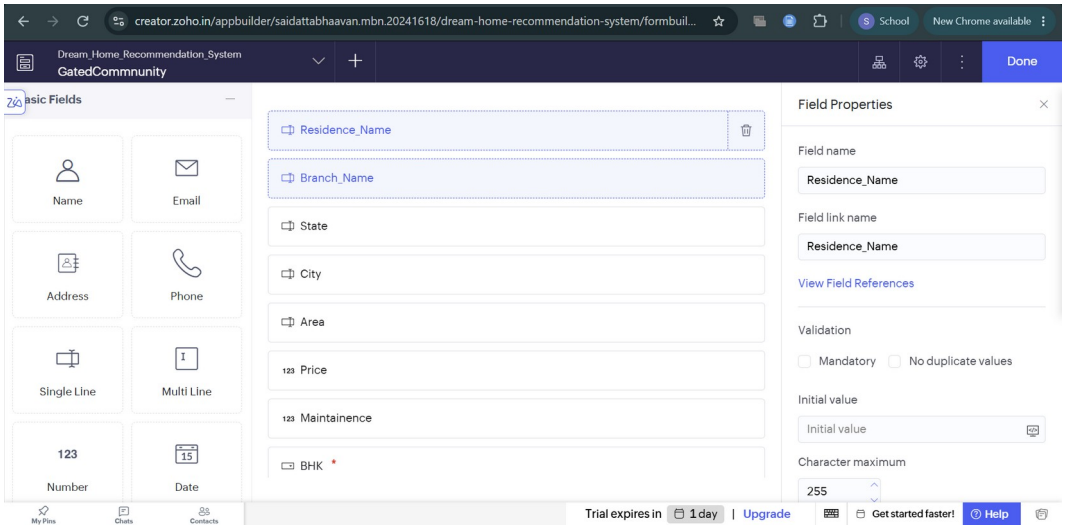


(c) Garbage to GatedCommnunity

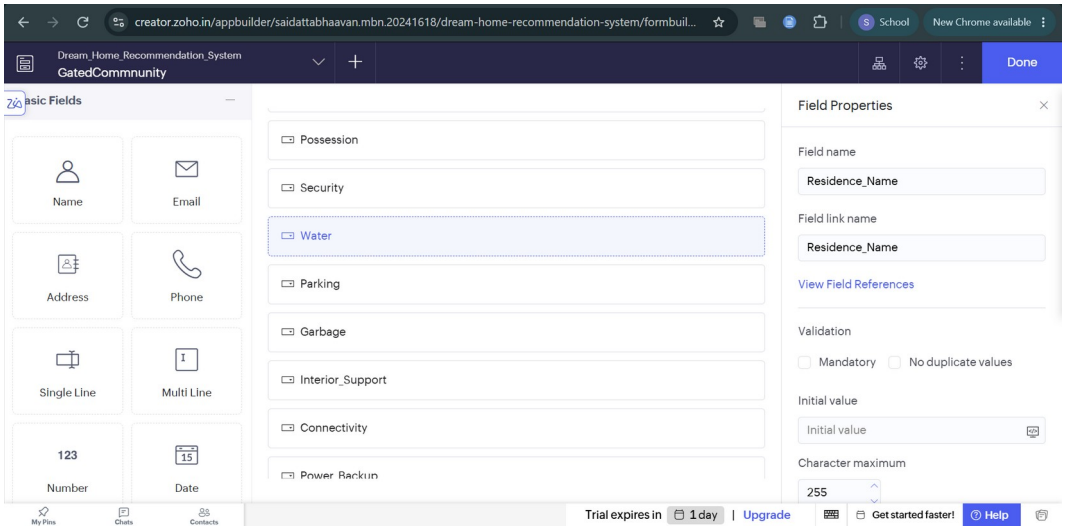


(d) Features_field1 to Result

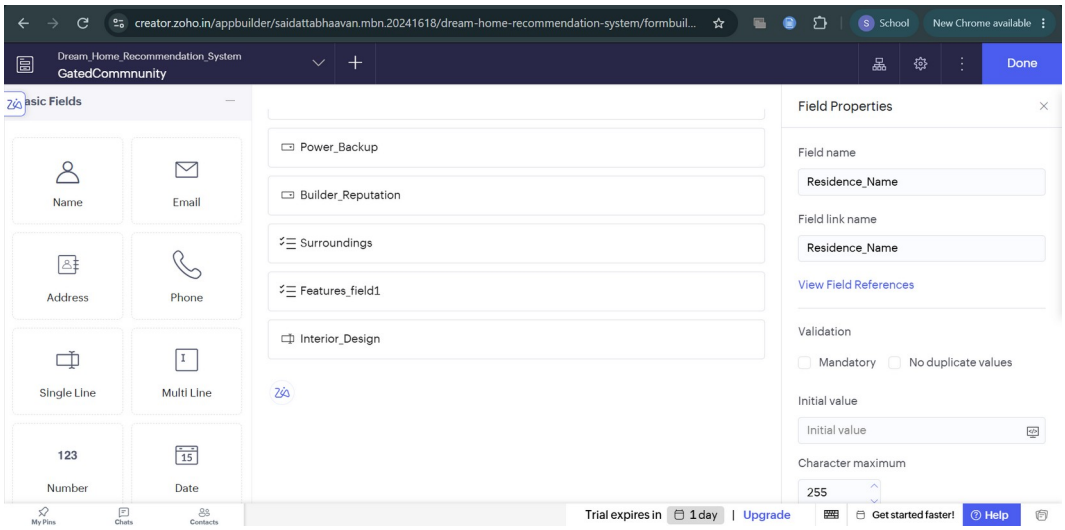
Figure 5.2: Buyer_Preference Form in the Form Builder



(a) Residence_Name to BHK

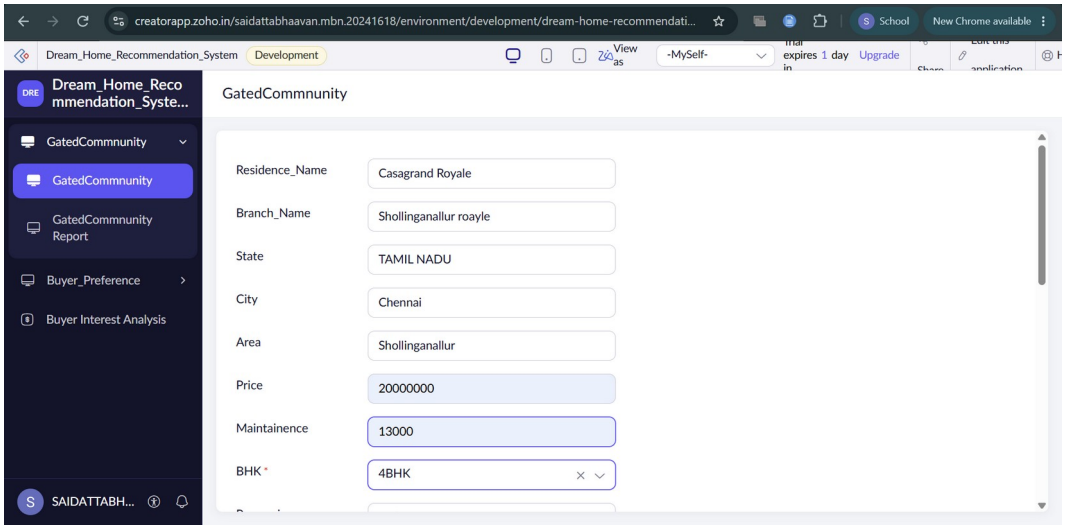


(b) Possession to Power_Backup

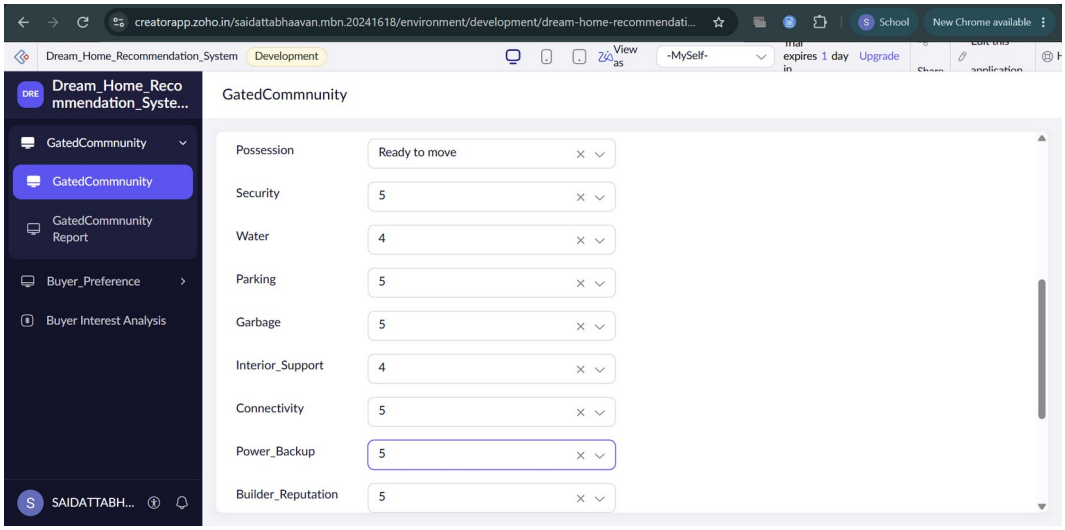


(c) Power_Backup to Interior_Design

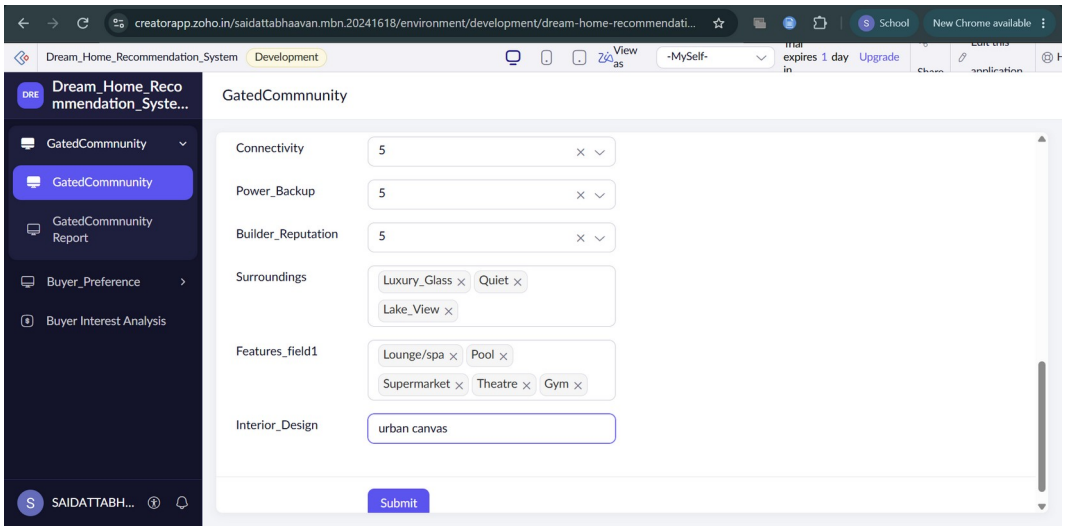
Figure 5.3: GatedCommnunity Form in the Form Builder



(a) Residence, location, price, maintenance and BHK



(b) Possession and ratings



(c) Surroundings, features, interior design and Submit

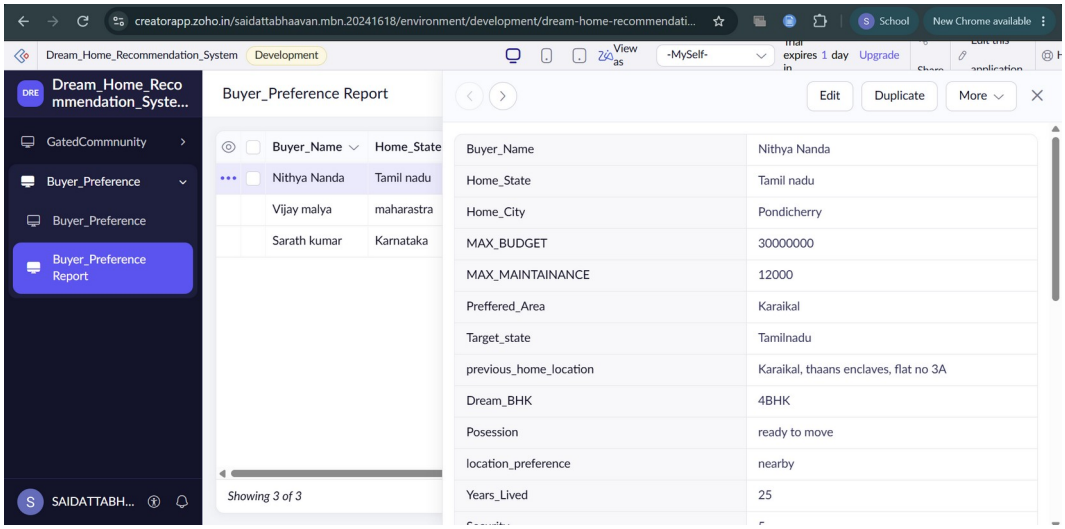
Figure 5.4: GatedCommnunity Form in Live Mode

5.3 Reports

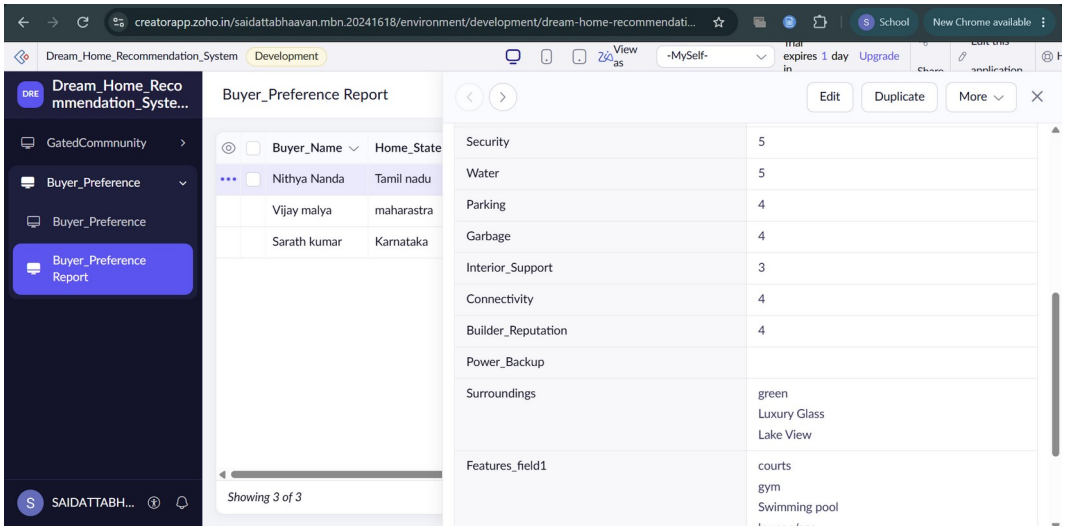
One list report was generated for each form: Buyer_Preference Report and GatedCommnunity Report. A list report shows one row per record and opens a detail panel with all fields when a row is clicked; the panel has Edit, Duplicate and More actions and arrows to move to the previous or next record. Table 5.1 summarises the reports.

Table 5.1: Reports Implemented

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
Buyer_Preference Report	List	Buyer_Preference	Default columns; record detail panel with all 25 fields; previous / next navigation	Edit, Duplicate, More (default record actions)	Administrator
GatedCommnunity Report	List	GatedCommnunity	Default columns; record detail panel with all 20 fields	Edit, Duplicate, More (default record actions)	Administrator



(a) Record detail panel – buyer details and previous-home location



(b) Record detail panel – previous-home ratings, surroundings and features

Figure 5.5: Buyer_Preference Report with the Record Detail Panel

5.4 Pages and Dashboards

The Buyer Interest Analysis page was built with the page (UI) builder. It reads directly from the application's forms and contains two charts: a horizontal bar chart of the price of each gated community and a pie chart of the Result field of the buyer records, which shows how many buyers fall into each recommendation category (Figure 5.6). Because the charts read live data, they change as soon as a record is added or recalculated.

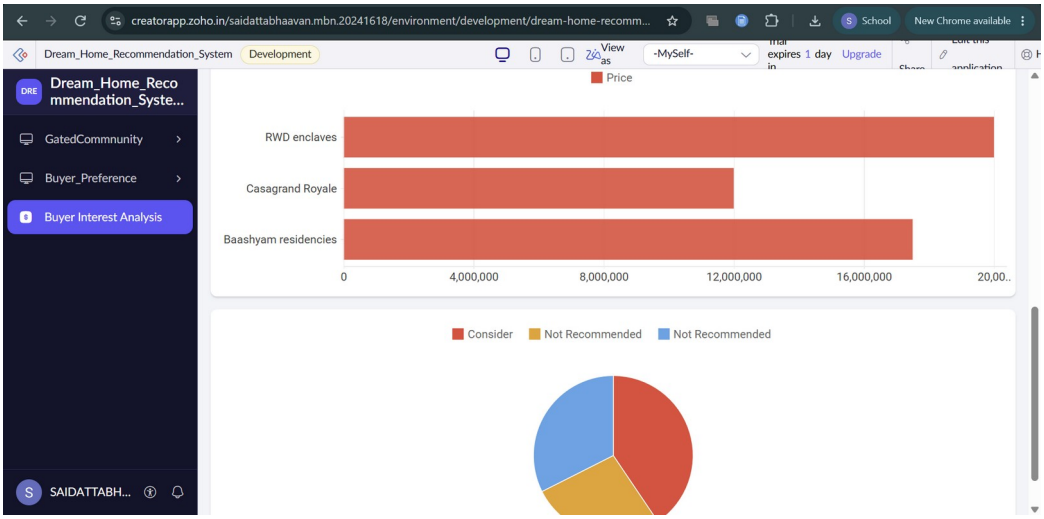


Figure 5.6: Buyer Interest Analysis Page (Bar Chart of Prices and Pie Chart of Results)

5.5 Deluge Scripting

The application has one Deluge script, DS-01, attached to the workflow Calculate Recommendation. It runs after a Buyer_Preference record is created or edited and its submission succeeds. Table 5.2 summarises the script, Codes 5.1 to 5.3 show its main parts, and the complete listing is given in Appendix C.

Table 5.2: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Buyer_Preference → Created or Edited → Successful form submission → Calculate Recommendation	Find the best-matching community for a buyer and store the community, final score and result	fetch records with criteria, for each loops, if / else if / else, ifnull, toDecimal, collection size(), contains, equalsIgnoreCase, round, record update

Code 5.1: *Importance of each factor (previous-home rating converted to a weight)*

```
// DS-01  |  Form: Buyer_Preference  |  Event: Created or Edited -> Successful
form submission
buyer = Buyer_Preference[ID == input.ID];
// importance of each factor = 6 - rating of the previous home (blank counts
as 3)
iSec = 6 - ifnull(input.Security,"3").toDecimal();
iWat = 6 - ifnull(input.Water,"3").toDecimal();
iPar = 6 - ifnull(input.Parking,"3").toDecimal();
iGar = 6 - ifnull(input.Garbage,"3").toDecimal();
iInt = 6 - ifnull(input.Interior_Support,"3").toDecimal();
iCon = 6 - ifnull(input.Connectivity,"3").toDecimal();
iPow = 6 - ifnull(input.Power_Backup,"3").toDecimal();
iBui = 6 - ifnull(input.Builder_Reputation,"3").toDecimal();
totalImp = iSec + iWat + iPar + iGar + iInt + iCon + iPow + iBui;
```

Code 5.2: *Fetching the catalogue, the budget filter and the location filter*

```
bestScore = 0.0;
bestId = 0;
found = false;
allComm = GatedCommnunity[ID != 0];
for each  c in allComm
{
    // 1. budget filter (price and monthly maintenance)
    if(c.Price <= input.MAX_BUDGET && c.Maintainence <=
input.MAX_MAINTAINANCE)
    {
        // 2. location filter
        locOk = true;
        pref = ifnull(input.location_preference,"").toString();
        cCity = ifnull(c.City,"").toString();
        cState = ifnull(c.State,"").toString();
        if(pref == "Nearby")
        {
            locOk =
cCity.equalsIgnoreCase(ifnull(input.Home_City,"").toString());
        }
        else if(pref == "Same state")
        {
            locOk =
cState.equalsIgnoreCase(ifnull(input.Home_State,"").toString());
        }
        else if(pref == "Other state")
        {
            locOk =
cState.equalsIgnoreCase(ifnull(input.Target_state,"").toString());
        }
        if(locOk)
```

Code 5.3: *Final score, best-community selection and result band*

```
            // 6. final score (capped at 5)
            finalScore = 0.7 * expScore + 0.3 * dreamScore + bonus;
            if(finalScore > 5)
            {
                finalScore = 5.0;
            }
            if(found == false || finalScore > bestScore)
            {
                found = true;
                bestScore = finalScore;
                bestId = c.ID;
            }
        }
    }
}
if(found)
{
    res = "Not Recommended";
    if(bestScore >= 3.75)
    {
        res = "Highly Recommended";
    }
    else if(bestScore >= 3.0)
    {
        res = "Consider";
    }
    buyer.GatedCommnunity=bestId;
    buyer.Final_score=bestScore.round(2);
    buyer.Result=res;
}
else
{
    buyer.Final_score=0;
    buyer.Result="Not Recommended";
}
```

How the script works. First, the buyer record is read and each previous-home rating is converted into an importance value, 6 minus the rating, so that a factor rated 1 in the old home gets importance 5 and a factor rated 5 gets importance 1; a blank rating is treated as 3. The importance values are added into totalImp. Second, all communities are fetched and examined one by one. A community is skipped unless its price is within the buyer's budget and its maintenance within the maintenance limit. The location preference then decides whether the community's city (Nearby), its state (Same state) or the buyer's target state (Other state) must match; the comparison ignores upper and lower case.

For every community that passes, the experience score is the importance-weighted average of the community's ratings, so factors that the buyer found lacking in the old home count more. The dream-match score is 5 × (0.2 × BHK match + 0.2 × possession match + 0.3 × share of preferred features found + 0.3 × share of preferred surroundings found). A bonus of 0.3 is added if the preferred area equals the community's area. The final score is 0.7 × experience score + 0.3 × dream score + bonus, limited to 5, and the

community with the highest score is remembered. After the loop the score is rounded to two decimals and converted into a result: Highly Recommended for 3.75 or more, Consider for 3.0 or more, otherwise Not Recommended. If no community passed the filters, the score is set to 0 and the result to Not Recommended.

5.6 Workflows and Automation

The application uses one form workflow. It is a workflow on the Buyer_Preference form that is triggered when a record is created or edited and the form submission is successful, and its only action is the Deluge script DS-01. All conditions (budget, maintenance and location) are evaluated inside the script, so no separate workflow criteria are needed. Figure 5.7 shows the workflow in the Deluge editor, with the path Buyer_Preference → Created or Edited → Successful form submission → Calculate Recommendation shown at the top.

Table 5.3: Workflows Implemented

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Calculate Recommendation	Form Workflow	Created or Edited – Successful form submission	None (filters are inside the script)	Update GatedCommnunity, Final_score and Result on the buyer record	DS-01

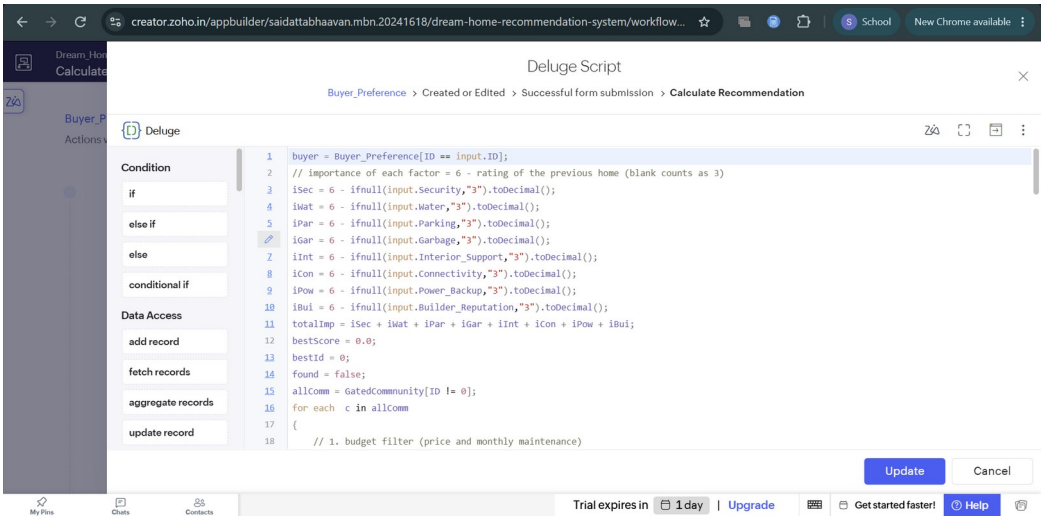


Figure 5.7: Calculate Recommendation Workflow in the Deluge Editor

5.7 Integrations

No integration with another Zoho service or an external API was implemented in version 1.0. The recommendation needs only the data stored in the two forms, so a connection, an invokeurl call or a webhook was not required, and no credential or API key is stored in the application. Table 5.4 records this explicitly. Possible integrations, such as e-mailing the recommendation to the buyer through Zoho Mail or pushing buyers as leads to Zoho CRM, are listed in Section 9.2.

Table 5.4: Integrations Implemented

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
None implemented	—	—	—	Not required for the scope of version 1.0

5.8 User Management and Role-Based Access

Version 1.0 is operated through the Administrator account of the application owner, which has full access to forms, reports, the page and the environments. No additional users, roles or permission profiles were configured, so the matrix in Table 4.4 is implemented only for the Administrator. The Home Buyer column of the matrix is the planned design for a future version in which buyers sign in themselves and see only their own records, for example with a record-level criterion such as Added User == zoho.loginuser. The Users section of the admin console is where such profiles would be created.

5.9 AI and Analytics Features

No Zia or other AI feature was used in version 1.0; the recommendation is produced by the explicit, explainable scoring rules of DS-01 rather than by a learned model, which makes every result reproducible by hand. Analytics are provided by the two charts of the Buyer Interest Analysis page (Figure 5.6). Because the scoring script was written and checked by the author, no part of the application logic is AI-generated.

5.10 Pro-Code Extension

No pro-code extension was needed. A Zoho Catalyst function or a JavaScript widget would become useful if the system were extended with machine-learning recommendations, travel-distance calculation between the buyer’s workplace and each

community, or large-scale data processing. For the present rule-based scoring over a catalogue of modest size, Deluge is sufficient.

5.11 Mobile and Portal Access

Zoho Creator generates web, tablet and phone layouts for the forms, reports and pages of an application, and the same application can be opened in the Zoho Creator mobile app after the user signs in. Mobile access and a customer portal were not part of the demonstrated scope of version 1.0, so no mobile screenshots are included.

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing was done at three levels. Unit testing checked each form by creating and editing records and viewing them in the report. Script testing checked DS-01 by saving buyer records and reading the Final_score, Result and GatedCommnunity values written back to the record, including the case in which no community qualifies. The report, the lookup and both charts were exercised with three sample buyer records and a catalogue of several sample communities. Deployment testing checked that version 1.0 could be published to the Stage and Production environments. Role-based access testing was not applicable because only the Administrator account exists in version 1.0.

6.2 Test Cases

Table 6.1 lists the test cases. Test cases marked “To run” have a defined input and expected result but their actual result has not yet been recorded; they should be executed on the live application and the table updated before submission.

Table 6.1: Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-03	Save a buyer record with budget, maintenance, location and ratings	Create the record for Nithya Nanda (budget 30,000,000 ; maintenance 12,000)	Record is saved and appears in Buyer_Preference Report	Record saved; report shows 3 of 3 records	Pass
TC-02	FR-01	Save a community record with BHK	Create the record for Casagrand Royale (price 20,000,000 ; maintenance 13,000; 4BHK)	Record is saved and appears in reports and charts	Record saved; community appears in the dashboard bar chart	Pass
TC-03	FR-01	Submit GatedCommnunity without the mandatory BHK	Leave BHK empty and submit	Validation error; record not saved	To run	To run

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-04	FR-05, FR-06, FR-10	No community satisfies the filters	Nithya Nanda: Nearby, home city Pondicherry, maintenance limit 12,000	Final_score = 0 and Result = Not Recommended	Final_score 0.00 and Result Not Recommended (Figure 8.2)	Pass
TC-05	FR-05	Maintenance limit below every community	Set MAX_MAINTAINANCE below the lowest community maintenance	Final_score = 0; Not Recommended	To run	To run
TC-06	FR-05	Budget below every community price	Set MAX_BUDGET below the lowest price	Final_score = 0; Not Recommended	To run	To run
TC-07	FR-06	Location preference Nearby	Home_City equal to a community City	That community is considered and may be recommended	To run	To run
TC-08	FR-06	Location preference Same state	Home_State equal to a community State (case ignored)	That community is considered	To run	To run
TC-09	FR-06	Location preference Other state	Target_state equal to a community State	That community is considered	To run	To run
TC-10	FR-07	Blank previous-home ratings	Leave all eight ratings blank	Each importance is 3 and the record is still scored	To run	To run
TC-11	FR-08	Preferred area bonus	Preffered_Area equal to the community Area	Final score is 0.3 higher than without the match (unless capped at 5)	To run	To run

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-12	FR-09	Result band Highly Recommended	Choose inputs giving a final score of 3.75 or more	Result = Highly Recommended	To run	To run
TC-13	FR-09	Result band Consider	Choose inputs giving a final score from 3.0 to 3.74	Result = Consider	The pie chart shows a Consider slice (Figure 5.6)	Pass
TC-14	FR-09	Edit a saved buyer record	Change MAX_BUDGET and save	Score and result are recalculated	To run	To run
TC-15	FR-09	Lookup is filled with the best community	Save a buyer record that has a qualifying community	GatedCommnunity shows the best-scoring community	To run	To run
TC-16	FR-12	Dashboard reflects a new record	Add a buyer record and reopen Buyer Interest Analysis	Pie chart counts the new result	To run	To run
TC-17	FR-12	Open the dashboard page	Open Buyer Interest Analysis	Bar chart and pie chart are displayed	Both charts displayed (Figure 5.6)	Pass
TC-18	FR-13	Publish version 1.0	Publish from Development to Stage and Production	Version 1.0 appears in both environments	Stage 1.0 and Production 1.0 dated 01 Nov 2026 (Figure 7.1)	Pass

6.3 Debugging Deluge Scripts

The main debugging technique was to inspect the values that the script writes back to the buyer record. After each save, the record detail panel of the report shows GatedCommnunity, Final_score and Result, which exposes whether the filters, the

scoring and the result band behaved as expected, and the importance weights can be re-computed by hand from the previous-home ratings. The Deluge editor also allows info statements to print intermediate values such as totalImp, expScore and bestScore, and the workflow log can be used to see run-time errors; no info statements were left in the final script, so the stored fields are the evidence used in this report.

The most useful finding from this technique was that a record can show a community name together with a score of 0.00 and the result Not Recommended (Figure 8.2). The else branch of the script resets only Final_score and Result, and does not clear the GatedCommnunity lookup, so a value saved earlier remains. This is recorded as defect BUG-02 in Table 6.2.

6.4 Defect Log

Table 6.2 lists the defects found while examining the stored data. They concern data consistency, naming and unused data rather than application errors, and none prevents the recommendation from running. The proposed fixes are described; the status is Open until they are applied.

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-01	Choice values of Surroundings and Features_field1 differ between the two forms (for example “Swimming pool” and “Pool”, “Luxury Glass” and “Luxury_Glass”).	DS-01 matches buyer choices against the community text with contains(), so differently worded or spelled values never match and the dream-match score is understated.	Proposed: use identical choice lists in both forms (or one shared source).	Open
BUG-02	A buyer record with Final_score 0.00 and Result Not Recommended still shows a community name in the GatedCommnunity field.	The else branch of DS-01 does not clear the lookup field; an earlier value remains.	Proposed: add buyer.GatedCommnunity = null; to the else branch.	Open

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-03	Typing differences in text locations, such as “Tamil nadu” and “Tamilnadu”, are treated as different states.	Home_State, Target_state, State and City are single-line fields compared with equalsIgnoreCase().	Proposed: change these fields to drop-downs with a fixed list.	Open
BUG-04	The legend of the result pie chart lists “Not Recommended” twice.	Possible duplicate or trailing-space variant in the Result choices or the chart grouping.	Proposed: check the Result choice list and the chart’s group-by field.	Open
BUG-05	Field and value names are inconsistent: Posession (Buyer_Preference) and Possession (GatedCommnunity), Maintainence, MAX_MAINTAINANCE, GatedCommnunity and the branch name “Shollinganallur roayle”.	Typing errors made while naming fields and entering sample data.	Proposed: correct the labels and link names (script references must be updated together) and the sample value.	Open
BUG-06	Years_Lived, previous_home_location, Branch_Name and Interior_Design are captured but not used by DS-01.	These fields were added for information; the scoring rules do not read them.	Proposed: use them in a later version (for example Years_Lived as a weight on the ratings) or remove them.	Open

6.5 User Acceptance Testing

User acceptance feedback from at least two end users has to be recorded in Table 6.3 after they use the application in their assigned role.

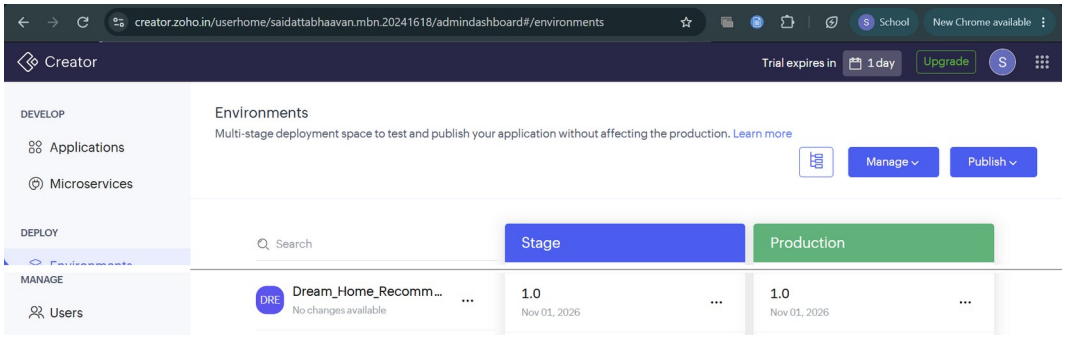
Table 6.3: User Acceptance Feedback

User	Role Tested	Feedback	Action Taken
[User 1 name]	[Role tested]	[Feedback]	[Action taken]
[User 2 name]	[Role tested]	[Feedback]	[Action taken]

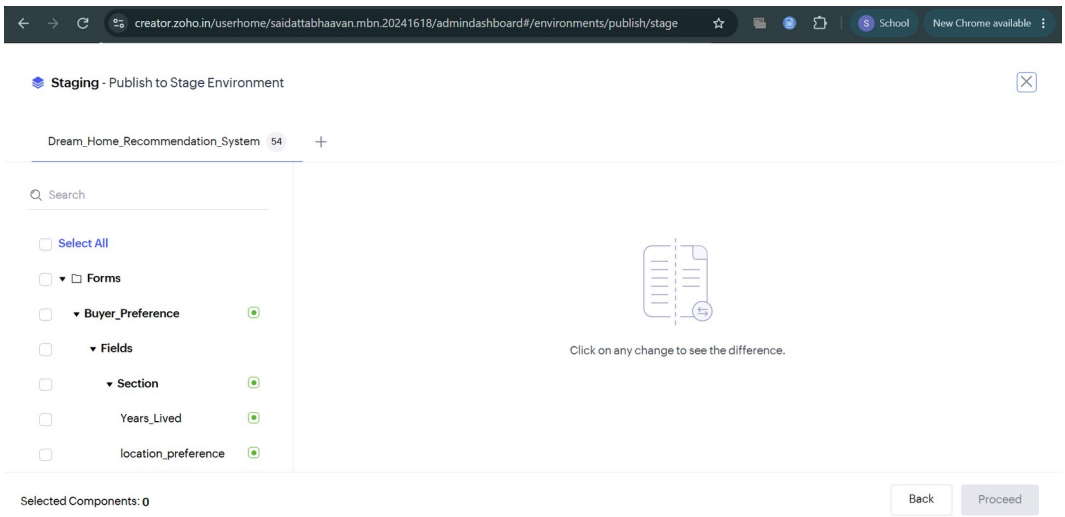
CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

The application was developed and tested in the Development environment of the author's Zoho Creator trial account. Zoho Creator provides a multi-stage deployment space in which an application is published first to the Stage environment for testing and then to the Production environment without affecting live users. Version 1.0 of Dream_Home_Recommendation_System was published to Stage and to Production on 01 November 2026, as shown on the Environments page (Figure 7.1(a)). The publish dialog lists the components that will be released, such as forms, fields, reports and the page, and lets each change be reviewed before it is sent (Figure 7.1(b)). Any later change will be released as a new version after the checklist in Table 7.1 is completed.



(a) Deploy → Environments: Dream_Home_Recommendation_System version 1.0 in Stage and Production



(b) Publish to Stage: components and fields selected for release

Figure 7.1: Environments and Publish to Stage

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done (✓)
1	All test cases executed in Development	Partly – see Table 6.1
2	Sample/test data removed or isolated	Sample data (3 buyer records and the community catalogue) kept for demonstration
3	Roles, permissions and users configured in Production	Not applicable (Administrator only)
4	Connections authorised in Production	Not applicable (no connections)
5	Workflow Calculate Recommendation enabled	✓
6	Application published to Stage and Production and URL verified	✓
7	Version / release notes recorded	✓ Version 1.0 (01 Nov 2026)

7.2 Performance Optimisation

The scoring script was written with performance in mind, although no timing measurements were taken. The importance weights depend only on the buyer, so they are computed once before the loop rather than for every community. All communities are fetched with a single statement instead of fetching inside the loop. The filters are ordered so that the cheapest checks (price and maintenance) run first and the more expensive scoring is done only for communities that pass them. Table 7.2 summarises these choices; the Before and After columns describe the design, not measured times.

Table 7.2: Performance Optimisation Results

Optimisation Applied	Component	Before	After	Improvement
Importance weights computed once per buyer	DS-01 (before the loop)	Recomputed for each community	Computed once	Fewer calculations per run
Single fetch of the catalogue	DS-01	Fetch inside the loop	One fetch statement	Fewer database calls

Optimisation Applied	Component	Before	After	Improvement
Budget and maintenance checked first	DS-01 (filters)	Score every community	Score only communities that pass	Less scoring work

7.3 Security Features

Version 1.0 relies on the platform's built-in security. Access requires a Zoho account sign-in, and the application is operated through the owner's account. The script contains no password, token or API key, and the application makes no external calls. The separation of Development, Stage and Production keeps untested changes away from live data. Finer controls such as roles, record-level criteria and portal security are listed as future work in Section 9.2.

Table 7.3: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
Zoho account sign-in	Application access	Unauthorised access
Separate Development, Stage and Production environments	Deployment (Section 7.1)	Untested changes reaching live data
No credentials or API keys in scripts	DS-01	Credential leakage
Mandatory BHK and drop-down choice lists	Forms	Invalid or inconsistent input

7.4 Governance and Administration

The publish view acts as a change log. For every added or modified component it shows the component type and name, the user who made the change and the date and time; for example, the page Buyer_Interest_Analysis is recorded as added by the author on 01-Nov-2026 at 15:25:23 (Figure 7.2). Each record also keeps Zoho's system fields for the user and time of creation. Components follow a consistent naming convention based on the form names, and the version numbers in the Environments page identify what is live.

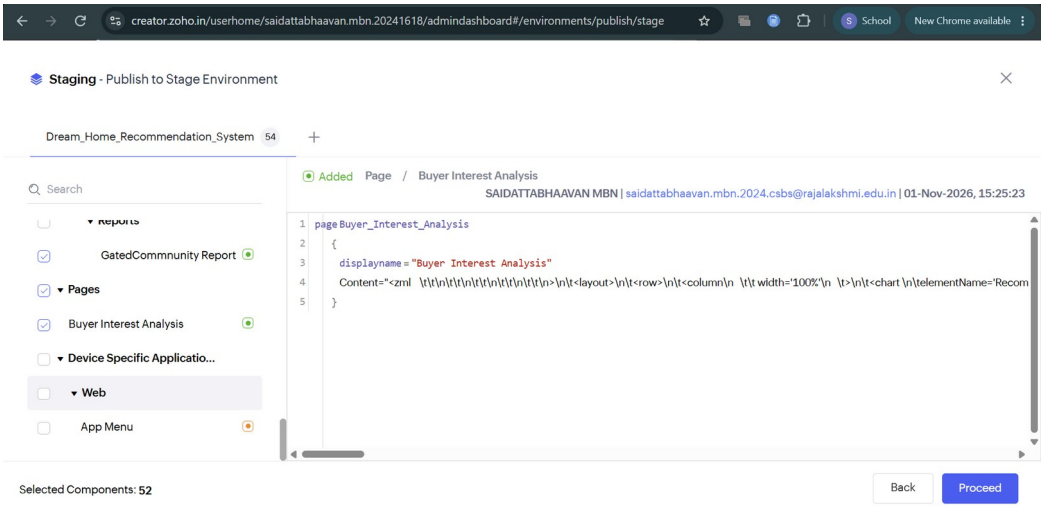


Figure 7.2: Component Change Log in the Publish to Stage View

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

Scenario. The author enters a buyer, Nithya Nanda, who lives in Pondicherry (state entered as “Tamil nadu”) and wants a ready-to-move 4BHK home nearby, preferably in Karaikal. Her budget is 30,000,000 and her maximum monthly maintenance is 12,000. She has lived for 25 years in a flat in Karaikal and rates it Security 5, Water 5, Parking 4, Garbage 4, Interior_Support 3, Connectivity 4 and Builder_Reputation 4, leaving Power_Backup blank. She would like green and lake-view surroundings with a gym, courts and a swimming pool (Figure 8.1).

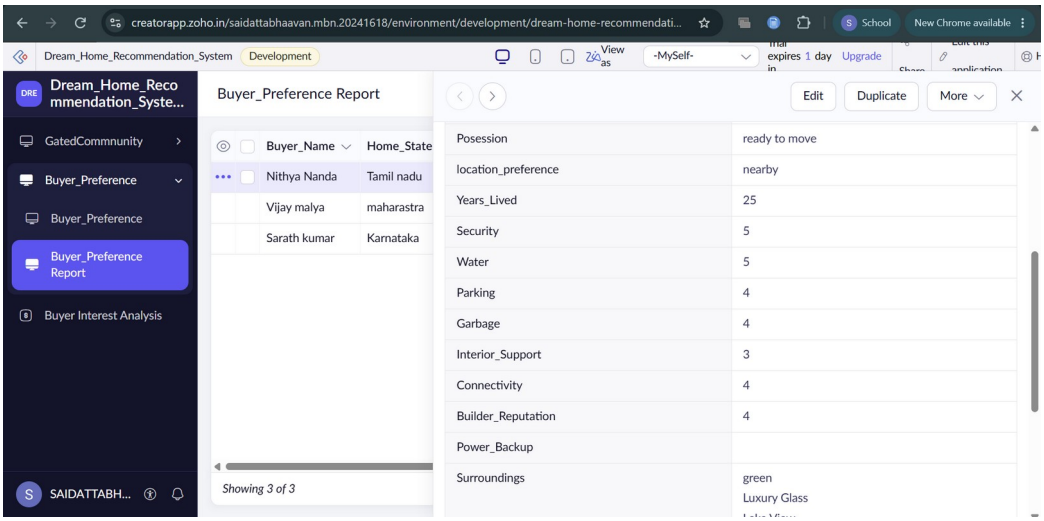


Figure 8.1: Scenario Step 1 – Buyer Preferences and Previous-Home Ratings Stored in the Report

When the record is saved, DS-01 converts her ratings into importance values: Security 1, Water 1, Parking 2, Garbage 2, Interior_Support 3, Connectivity 2, Builder_Reputation 2 and Power_Backup 3 (blank counts as 3), so that totalImp is 16 and the factors her old home handled worst, interior support and power backup, carry the most weight. Each community is then checked against her budget, maintenance limit and location preference. The catalogue records seen in the application, for example Casagrand Royale with a maintenance of 13,000, do not satisfy all the filters, and the script reports that no community qualified: Final_score is 0.00 and Result is Not Recommended (Figure 8.2). This exercises the no-match branch of the script (FR-10).

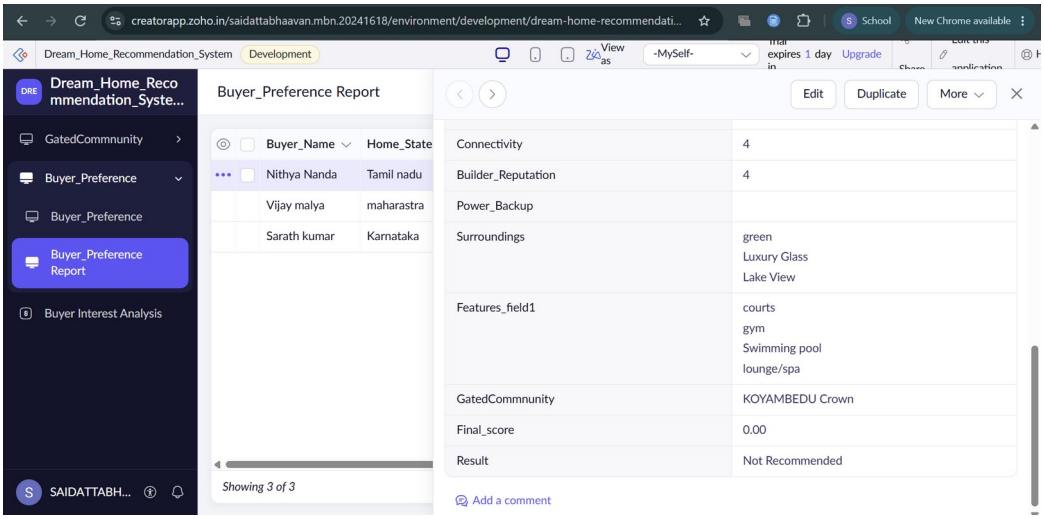


Figure 8.2: Scenario Step 2 – Result Fields Written by the Workflow (Final_score 0.00, Not Recommended)

The last two steps of the scenario are visible on the dashboard. The bar chart of prices lets the advisor see how the catalogue is priced, and the pie chart counts every buyer record by its Result, including the Consider and Not Recommended categories (Figure 5.6). The figure also shows that BUG-02 (a community name remaining beside a zero score) should be fixed before the application is used with real buyers.

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
Design a data model and two forms with consistent values	Sections 4.2, 4.3 and 5.2	GatedCommnunity (20 fields) and Buyer_Preference (25 fields) linked by a lookup; Figures 5.2 to 5.4
Automate the recommendation with a Deluge script in a form workflow	Sections 5.5 and 5.6	DS-01 filters, scores and ranks communities when a buyer record is saved; Appendix C
Store the best community, final score and result on the buyer record	Sections 5.5 and 8.1	Final_score and Result written back; observed in Figure 8.2
Provide reports and a dashboard	Sections 5.3 and 5.4	Two list reports and the Buyer Interest Analysis page with two charts; Figures 5.5 and 5.6
Deploy version 1.0 through Development, Stage and Production	Section 7.1	Version 1.0 in Stage and Production on 01 November 2026; Figure 7.1

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
Test with sample data and document defects	Sections 6.2 and 6.4	18 test cases defined, 5 with recorded results; 6 defects logged with proposed fixes

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application	Improvement
Comparing options for one buyer	Manual reading of brochures and notes	Automatic evaluation of every community when the record is saved	Faster and complete
Manual steps	Filtering by budget, maintenance and location by hand	Filters applied by the script	Fewer manual steps
Consistency of advice	Depends on the advisor	Same rules and weights for every buyer	Repeatable
Record of reasoning	None	Community, score and result stored on the record	Traceable
Summary reporting	Manual counting	Dashboard charts	Instant overview

8.4 Limitations of the Low-Code Approach

Several limits were met. The scoring logic is a rule-based formula; Zoho Creator has no built-in recommendation engine, so every weight and threshold is coded in Deluge and must be edited in the script. Matching of multi-select values is done by text comparison with contains(), which is sensitive to spelling and wording (BUG-01), and location matching compares typed text (BUG-03); the workaround is to use identical choice lists and drop-downs. The Deluge editor gives limited debugging support compared with an IDE, the dashboard has limited layout and styling options, and the application is tied to the Zoho platform (vendor lock-in). The trial edition used here also expires after a short period, so a paid edition would be needed for continued use.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

On the spectrum, the forms, the lookup relationship, the two list reports, the dashboard charts and the Stage / Production publishing were built with no-code configuration. The

recommendation script DS-01 is low-code: it needed Deluge for fetch, loops, conditions and string handling, but only about one hundred lines. A move to pro-code would be justified if the recommendation were to use machine learning on past buyer choices, road-distance calculation between a buyer’s workplace and each community, or very large catalogues; such logic could run in a Zoho Catalyst function called from the workflow.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The Dream Home Recommendation System shows how a recurring advisory decision, choosing a gated community for a home buyer, can be turned into a structured, repeatable process on a low-code platform. Two forms hold the community catalogue and the buyer requirements, one form workflow runs a Deluge script that filters and scores the communities using the buyer's previous-home experience, and two list reports and a dashboard page present the data. Version 1.0 was published to the Stage and Production environments on 01 November 2026.

The work also showed where care is needed in a low-code solution: consistent choice values between forms, clearing results when nothing matches, and fixed lists instead of typed text. These points are documented as defects with proposed fixes. The application is a sound base that can be extended with notifications, roles and external services.

9.2 Future Enhancements

- Correct BUG-01 to BUG-04 by aligning choice lists, clearing the lookup when no community matches, using drop-downs for state and city, and checking the result chart.
- Add Zoho Mail notifications that send the recommendation and score to the buyer.
- Create a Home Buyer profile and portal so that buyers enter their own preferences and see only their own records.
- Store the scoring weights and thresholds in a settings form so that they can be changed without editing the script.
- Add a scheduled workflow and report of buyers in the Consider category for follow-up.
- Use a Catalyst function or Zia to learn weights from past buyer choices and to calculate travel distance to the buyer's workplace.

9.3 Learning Outcomes and Career Relevance

Developing this application gave practical experience of data modelling with forms and lookups, Deluge scripting with loops, conditions and record updates, building reports

and pages, and publishing through environments. Checking the stored results also taught the importance of data consistency and of testing the no-result path of a script. These skills apply directly to roles such as low-code developer, Zoho Creator consultant, solutions consultant and citizen developer, and the habit of checking an algorithm by hand against the stored output applies to software development in general.

REFERENCES

[1] Government of India, “The Real Estate (Regulation and Development) Act, 2016,” Ministry of Housing and Urban Affairs, New Delhi, India, 2016.

[2] Zoho Corporation, “Zoho Creator Help Documentation.” [Online]. Available: https://www.zoho.com/creator/help/

[3] Zoho Corporation, “Deluge Script Reference Guide.” [Online]. Available: https://www.zoho.com/deluge/help/

[4] A. Sahay, A. Indamutsa, D. Di Ruscio and A. Pierantonio, “Supporting the understanding and comparison of low-code development platforms,” in *Proc. 46th Euromicro Conf. Software Engineering and Advanced Applications (SEAA)*, 2020, pp. 171–178.

[5] A. C. Bock and U. Frank, “Low-code platform,” *Business & Information Systems Engineering*, vol. 63, no. 6, pp. 733–740, 2021.

[6] D. Di Ruscio, D. Kolovos, J. de Lara, A. Pierantonio, M. Tisi and M. Wimmer, “Low-code development and model-driven engineering: Two sides of the same coin?,” *Software and Systems Modeling*, vol. 21, pp. 437–446, 2022.

[7] F. Ricci, L. Rokach and B. Shapira, Eds., *Recommender Systems Handbook*, 2nd ed. New York, NY, USA: Springer, 2015.

[8] R. L. Keeney and H. Raiffa, *Decisions with Multiple Objectives: Preferences and Value Tradeoffs*. Cambridge, U.K.: Cambridge Univ. Press, 1993.

[9] W. Rosam and O. Kalnichevski, *Low-Code Development: A Practical Guide*. New Delhi, India: BPB Publications.

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table 9.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	Y	Dream_Home_Recommendation_System; sections GatedCommnunity, Buyer_Preference and page Buyer Interest Analysis	5.1	Fig. 5.1
II	Forms with ≥ 8 distinct field types	N	6 field types used: Single Line, Number, Drop Down, Multi Select, Lookup, Decimal	5.2	Table 4.2
II	Lookup field (relationship between forms)	Y	Buyer_Preference. GatedCommnunity → GatedCommnunity	4.2, 5.2	Table 4.1
II	Subform (parent–child data)	N	Not required (one buyer record points to one community)	4.2, 5.2	—
II	Field validations / layout rules	Y	Mandatory BHK; constrained drop-down and multi-select choices	5.2	Tables 4.2, 4.3
II	List report with filter, grouping, custom action	Y	Buyer_Preference Report; GatedCommnunity Report (default record actions)	5.3	Table 5.1, Fig. 5.5
II	Kanban / Calendar / Timeline / Map report	N	Not used	5.3	—
II	Pivot table / chart report	Y	Bar chart of prices and pie chart of results on the page	5.4	Fig. 5.6
II	Page / dashboard built with UI Builder	Y	Buyer Interest Analysis	5.4	Fig. 5.6

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
II	Deluge – form events (validate / success / field actions)	Y	Calculate Recommendation (on successful form submission)	5.5	Codes 5.1–5.3
II	Deluge – functions, fetch, loops, collections	Y	fetch with criteria, for each loops, collection size(), string functions	5.5	App. C
II	Email / in-app notifications	N	Planned (Section 9.2)	5.5, 5.6	—
II	Form workflow	Y	Calculate Recommendation on Buyer_Preference	5.6	Table 5.3, Fig. 5.7
II	Scheduled workflow	N	Not used	5.6	—
II	Approval process	N	Not used	5.6	—
II	Blueprint	N	Not used	5.6	—
II	Batch / report workflow	N	Not used	5.6	—
II	Testing & debugging evidence	Y	18 test cases and 6 defects	6	Tables 6.1, 6.2
III	Integration with a Zoho service	N	Not required in version 1.0	5.7	—
III	Connection / REST API call (invokeurl)	N	Not required in version 1.0	5.7	—
III	Webhook or Creator REST API	N	Not required in version 1.0	5.7	—
III	Users, roles and permission profiles	N	Administrator account only; roles planned	5.8	—
III	Record-level access criteria / customer portal	N	Planned (Section 5.8)	5.8	—
III	Zia / AI feature	N	No AI feature used (Section 5.9)	5.9	—
III	Analytics (charts / Zoho Analytics)	Y	Dashboard charts	5.4	Fig. 5.6
IV	Performance optimisation applied & measured	N	Design-level choices only; not measured (Section 7.2)	7.2	Table 7.2

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
IV	Deployment across environments	Y	Development, Stage v1.0 and Production v1.0	7.1	Fig. 7.1
IV	Security controls (encryption, OAuth, permissions)	N	Platform defaults only (Section 7.3)	7.3	Table 7.3
IV	Governance (audit trail, administration)	Y	Publish change log; environment versions	7.4	Fig. 7.2
V	Pro-code extension (Catalyst / widgets / custom functions)	N	Not required (Section 5.10)	5.10	—
V	Low-code limitations & spectrum analysis	Y	Sections 8.4 and 8.5	8.4, 8.5	—
—	Mobile app / portal access	N	Not demonstrated (Section 5.11)	5.11	—

Total features implemented: 14 / 32

APPENDIX B – COURSE OUTCOME MAPPING

Table 9.2: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2	2 – Medium	
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6	3 – High	
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7–5.9	1 – Low	
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7	2 – Medium	
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3	2 – Medium	

APPENDIX C – COMPLETE DELUGE SOURCE CODE

C.1 Buyer_Preference form – workflow Calculate Recommendation (DS-01)

```
// Script ID : DS-01
// Location  : Buyer_Preference → Created or Edited → Successful form
submission → Calculate Recommendation
// Purpose   : Find the best-matching gated community, final score and result
for a buyer

buyer = Buyer_Preference[ID == input.ID];
// importance of each factor = 6 - rating of the previous home (blank counts
as 3)
iSec = 6 - ifnull(input.Security,"3").toDecimal();
iWat = 6 - ifnull(input.Water,"3").toDecimal();
iPar = 6 - ifnull(input.Parking,"3").toDecimal();
iGar = 6 - ifnull(input.Garbage,"3").toDecimal();
iInt = 6 - ifnull(input.Interior_Support,"3").toDecimal();
iCon = 6 - ifnull(input.Connectivity,"3").toDecimal();
iPow = 6 - ifnull(input.Power_Backup,"3").toDecimal();
iBui = 6 - ifnull(input.Builder_Reputation,"3").toDecimal();
totalImp = iSec + iWat + iPar + iGar + iInt + iCon + iPow + iBui;
bestScore = 0.0;
bestId = 0;
found = false;
allComm = GatedCommnunity[ID != 0];
for each  c in allComm
{
    // 1. budget filter (price and monthly maintenance)
    if(c.Price <= input.MAX_BUDGET && c.Maintainence <=
input.MAX_MAINTAINANCE)
    {
        // 2. location filter
        locOk = true;
        pref = ifnull(input.location_preference,"").toString();
        cCity = ifnull(c.City,"").toString();
        cState = ifnull(c.State,"").toString();
        if(pref == "Nearby")
        {
            locOk =
cCity.equalsIgnoreCase(ifnull(input.Home_City,"").toString());
        }
        else if(pref == "Same state")
        {
            locOk =
cState.equalsIgnoreCase(ifnull(input.Home_State,"").toString());
        }
        else if(pref == "Other state")
        {
            locOk =
cState.equalsIgnoreCase(ifnull(input.Target_state,"").toString());
        }
        if(locOk)
        {
            // 3. experience score (blank community rating counts as 3)
            expScore = (iSec * ifnull(c.Security,"3").toDecimal() + iWat *
ifnull(c.Water,"3").toDecimal() + iPar * ifnull(c.Parking,"3").toDecimal() +
iGar * ifnull(c.Garbage,"3").toDecimal() + iInt *
ifnull(c.Interior_Support,"3").toDecimal() + iCon *
ifnull(c.Connectivity,"3").toDecimal() + iPow *
ifnull(c.Power_Backup,"3").toDecimal() + iBui *
ifnull(c.Builder_Reputation,"3").toDecimal()) / totalImp;
            // 4. dream match score
            bhkM = 0.0;
            if(c.BHK == input.Dream_BHK)
            {
                bhkM = 1.0;
            }
            posM = 0.0;
```

```
            if(input.Posession == "Any" || c.Possession == input.Posession)
            {
                posM = 1.0;
            }
            featM = 1.0;
            if(input.Features_field1 != null && input.Features_field1.size() >
0)
            {
                cFeat = ifnull(c.Features_field1,"").toString();
                hit = 0;
                for each  f in input.Features_field1
                {
                    if(cFeat.contains(f))
                    {
                        hit = hit + 1;
                    }
                }
                featM = hit.toDecimal() / input.Features_field1.size();
            }
            surrM = 1.0;
            if(input.Surroundings != null && input.Surroundings.size() > 0)
            {
                cSurr = ifnull(c.Surroundings,"").toString();
                hit2 = 0;
                for each  s in input.Surroundings
                {
                    if(cSurr.contains(s))
                    {
                        hit2 = hit2 + 1;
                    }
                }
                surrM = hit2.toDecimal() / input.Surroundings.size();
            }
            dreamScore = 5 * (0.2 * bhkM + 0.2 * posM + 0.3 * featM + 0.3 *
surrM);
            // 5. small bonus if the preferred area matches
            bonus = 0.0;
            if(ifnull(input.Preffered_Area,"") != "" &&
ifnull(c.Area,"").equalsIgnoreCase(input.Preffered_Area))
            {
                bonus = 0.3;
            }
            // 6. final score (capped at 5)
            finalScore = 0.7 * expScore + 0.3 * dreamScore + bonus;
            if(finalScore > 5)
            {
                finalScore = 5.0;
            }
            if(found == false || finalScore > bestScore)
            {
                found = true;
                bestScore = finalScore;
                bestId = c.ID;
            }
        }
    }
}
if(found)
{
    res = "Not Recommended";
    if(bestScore >= 3.75)
    {
        res = "Highly Recommended";
    }
    else if(bestScore >= 3.0)
    {
        res = "Consider";
    }
    buyer.GatedCommnunity=bestId;
    buyer.Final_score=bestScore.round(2);
    buyer.Result=res;
```

```
}
else
{
    buyer.Final_score=0;
    buyer.Result="Not Recommended";
}
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

Open the application link given in Project at a Glance in Google Chrome, Microsoft Edge or Mozilla Firefox, sign in with your Zoho account and choose Dream_Home_Recommendation_System. On a phone, install the Zoho Creator app, sign in with the same account and open the application from the list. The left menu has the sections GatedCommnunity and Buyer_Preference and the page Buyer Interest Analysis.

D.2 Instructions for the Administrator / Advisor – Maintaining the Community Catalogue

- Open GatedCommnunity → GatedCommnunity to see the form.
- Enter the residence name, branch name, state, city, area, price and monthly maintenance; choose the BHK (mandatory) and possession.
- Choose the ratings from 1 to 5 for security, water, parking, garbage, interior support, connectivity, power backup and builder reputation.
- Select the surroundings and features and enter the interior design, then click Submit.
- Open GatedCommnunity Report to view, edit or duplicate community records.

D.3 Instructions for Recording a Buyer and Reading the Recommendation

- Open Buyer_Preference → Buyer_Preference.
- Enter the buyer name, maximum budget, maximum maintenance, home state and city, preferred area, dream BHK and possession.
- Choose the location preference (Nearby, Same state or Other state); for Other state also enter the target state.
- Enter the previous-home location, years lived and ratings from 1 to 5 for the eight factors, then select the surroundings and features.
- Click Submit. The workflow calculates the recommendation automatically; leave the GatedCommnunity, Final_score and Result fields untouched.
- Open Buyer_Preference Report and click the buyer’s row to read the recommended community, the final score and the result.

- Open Buyer Interest Analysis to see the price chart and the result distribution of all buyers.